SemRec: A Real-Compiler-Grounded Divergence Oracle for Cross-Language Migration (C→Rust/Go/Swift/C++/OCaml/Zig/WebAssembly and Rust→C) via σ-Bridge Product Programs

halley-labs

Abstract

Migrating C to a safety-checked language or runtime (Rust, Go, Swift, Zig, WebAssembly) demands tooling that compares *source-language* semantics rather than relying on runtime agreement or LLVM-level normalization. The central hazard is *undefined-behavior (UB) divergence*: a C input on which the program's result is not fixed by the standard (the optimizer may resolve it either way, or a sanitizer traps), while the safe-language port computes one deterministic, defined value. A single dynamic run cannot expose this gap. We present SemRec, a repository-backed divergence oracle that encodes C and target semantics separately through a σ*-bridge product program*, finds a witnessing input or bounded execution with Z3, dataflow, or enumeration, and—crucially—*confirms every witness against real compilers* (`clang` under UndefinedBehaviorSanitizer, MemorySanitizer where available, `-Wunsequenced`, or a checked compiler-mode delta; `rustc`; `go`; and `zig`) so that no verdict ships unless an independent ground-truth harness confirms it. The oracle framework is a uniform plugin interface: each divergence class is an additive plugin, and a class automatically transfers across language pairs. The checked-in artifact implements a catalogue of 24 divergence classes—signed overflow, shift/division UB, `INT_MIN/-1`, out-of-bounds access, strict-aliasing (type-punning), sequence-point/unsequenced read-write UB, floating-point contraction and `-ffast-math` reassociation, VLA-bound, float→int overflow, pointer-provenance / address-space-overflow arithmetic, `restrict`-aliasing, overlapping `memcpy` vs. memmove-like slice copy, uninitialized struct-padding serialization vs. zero-padded safe serialization, `longjmp` to an exited VLA scope vs. structured unwinding, a scoped relaxed-atomics store-buffering ordering gap, an uninitialized-read/definedness analysis, two implementation-defined classes (bit-field packing and out-of-range `enum` representation), and a C/C++-boundary class (`1 « 31`, undefined in C yet defined in C++20)—driven across seven target languages (Rust, Go, Swift, C++, a GC'd, exception-based OCaml port, and Zig `ReleaseSafe`, plus WebAssembly value-or-trap semantics under `wasmtime`) by a deterministic, byte-reproducible cross-pair regression matrix whose toolchain-backed cells are *all* confirmed against real compilers/runtimes (`clang`, `rustc`, `go`, `clang++`, `ocamlopt`, `zig`, `wasmtime`) plus an executable checked-libc contract build for preconditions UBSan does not cover and an MSan/auto-init padding confirmation for indeterminate object bytes. The framework is not limited to C-rooted undefined behaviour in the forward direction: a *Rust→C reverse pair* proves that a Rust-defined panic can become target-side C UB under FFI-style lowering, and a first *safe→safe* pair, Go→Rust—in which *neither* side has any undefined behaviour—catches the *defined-but-different* hazard where `INT_MIN/-1` is a defined modular value in Go yet a guaranteed panic in Rust, raising the matrix to **60 cells over 9 pairs**, with confirmation delegated to the real compiler/toolchain/runtime for every available cell. The new C→WebAssembly column contributes four runtime-backed integer cells: C UB vs. wasm wrapping, shift-mask, division-by-zero trap, and signed-division-overflow trap semantics. To make the breadth claim falsifiable, a second cross-pair generalization study replays the registry over the newly added pairs/directions and requires every available positive witness to confirm while every safe control remains unflagged; an executable per-pair soundness checker separately verifies the registry entry, confirmation mode, target semantics pack, positive witness, negative control, and live confirmation for C→Zig, C→C++20, Rust→C, C→WebAssembly, Go→Rust, and C→OCaml. Beyond pairwise checks, an N*-language consistency oracle* compiles one C source to ≥ 3 safe targets at once and flags the minority target whose live output disagrees, and a Lean typeclass SPI contract plus a target-pack conformance suite mechanically certify that every back end satisfies the plugin/pack obligations. For the integer-UB family we additionally ship an SMT-backed encoding whose equivalence to a human-readable specification is Z3-proved for *every* 32/64-bit input, whose operational decision path agrees with an independent enumeration predicate on a deterministic 10,000-case differential harness, and which finds the rare overflow witnesses (e.g. `INT_MIN/-1`, one input in 2^{64}) in milliseconds where ordered enumeration exhausts its budget. For floating-point contraction, the artifact now retains the exact binary64 operand and result bit patterns and replays them through Z3's IEEE-754 FP theory before the real `clang`/ `rustc` confirmation path runs. An optimization-level sweep additionally pins the exact `-O` level at which each no-sanitizer-traps UB class first changes the program's observable output. On a labelled divergence study (29 cases over 19 anchor classes) the oracle attains perfect precision and recall ($\mathrm{tp} = 23,\ \mathrm{fp} = 0,\ \mathrm{fn} = 0,\ \mathrm{tn} = 528$) where a 200,000-trial differential fuzzer finds nothing on the UB-invisible classes. A Tier-1 c2rust-output corpus separately runs the real `c2rust 0.22.1` translator on 12 real-library extraction units and checks the generated Rust artifacts against the verifier. The same artifact also keeps the tool honest at semantic frontiers: volatile/inline-assembly/FFI/general atomic units cause loud abstention justified by real `clang` IR, while the scoped atomics oracle uses a deterministic model-level proof; a ThreadSanitizer/`go run -race`/`rustc` concurrency oracle proves that the same unsynchronized counter is C data-race UB, Go racy-but-defined, and rejected by Rust's safe `Send`/`Sync` discipline while synchronized repairs compile and run clean. We give a mechanized-friendly soundness development (σ-bridge coercion correctness, product-program isolation, a bounded-completeness boundary, Lean-checked class lemmas for strict-aliasing optimizer-exploitation and pointer-provenance trap-vs-defined witnesses, and an end-to-end theorem that counterexample packaging preserves the raw product-run witness), plus an independent Coq cross-check of the same core recorded-observable/product-witness theorem, a Lean-checked preservation theorem for the counterexample minimizer, and proof-carrying counterexamples: each confirmed source-UB witness serializes a canonical, hash-bound certificate over the recorded $P \wedge T \wedge C$ observation; the verdict-layer preimage is length-prefixed and Lean-proved injective, so canonicalization itself cannot merge distinct verdict payloads before SHA-256. An offline checker validates the certificate before a Lake-built checker re-validates the final `oracle_sound` inference. The artifact regenerates every headline number from content-hashed real-compiler runs, and a new soundness-regression gate enumerates all 60 registered oracle instances across 9 pairs, rejecting any plugin that lacks a per-pair soundness statement, matching confirmation mode, source-definedness premise, theorem/claim reference, and live witness probe; a generated soundness compendium maps every oracle instance to its theorem references and concrete witness unit.

1 Introduction

The software industry is undergoing a structural shift toward memory-safe languages. CISA's *Secure by Design* pledge [1] calls on software manufacturers to adopt memory-safe languages; the NSA has issued formal guidance recommending organizations transition from C/C++ to languages such as Rust [2]; and the White House Office of the National Cyber Director has endorsed memory safety as a national security priority [3]. Major codebases are responding: the Chromium project reports that 70% of high-severity security bugs stem from memory safety violations [4], and Microsoft attributes a similar fraction to the same root cause [5]. The Linux kernel now accepts Rust modules [6], and Android has seen a measurable reduction in memory-safety vulnerabilities as the fraction of new code written in Rust has increased.

The invisible-bug problem. The most dangerous C→Rust translation bugs are *provably invisible* to every existing verification approach. Signed integer overflow is undefined behavior (UB) in C (C11 §6.5/5 [10]) but wraps modulo 2^w in Rust release builds; division of `INT_MIN` by -1 is UB in C but wraps in Rust; shifts by $\geq w$ are UB in C but mask the shift amount in Rust. These divergences are *erased* when compiled to LLVM IR: the C compiler encodes "no signed wrap" as an `nsw` flag, while Rust omits it, but both produce the same `add i32` instruction. Moreover, on x86 hardware, C signed overflow *wraps identically* to Rust, so differential testing—even with edge-case-aware input generation—sees identical outputs and reports no divergence.

In our IR erasure experiment, 100% of UB divergences produced identical LLVM IR. In our differential testing experiment, UB-aware testing with 10,000 inputs *per pair* detected **0/30** UB divergences. IR-level tools (KLEE [11], SMACK [12], Alive2 [14]) also detect **0/30**. These bugs hide in every test suite, survive code review, and only surface when compiled with a different optimizer or on a different architecture.

Approach. SemRec addresses this gap via *σ-bridge product programs.* Given a C function f_C and its Rust translation f_R, SemRec:

1. Parses both independently (tree-sitter with recursive-descent fallback) and lowers to a shared typed SSA IR annotated with per-language overflow, shift, and division UB semantics.

2. Constructs a product program with shared symbolic inputs and disjoint variable namespaces, inserting σ-bridge coercion nodes at every program point where the semantic configurations σ_C and σ_R disagree.
3. Encodes the product program in QF_BV and discharges via Z3 [17]. C UB semantics become SMT assumptions (`assert_assume`); Rust wrapping semantics become concrete bitvector operations. UNSAT certifies bounded equivalence; SAT yields a counterexample; assumption violation yields a UB divergence witness.

Key result. On 30 IR-invisible UB divergences spanning 6 classes (overflow, shift, division, cast, negate, compound) plus 10 equivalent negative controls, SemRec achieves **100% recall** (30/30 divergences detected), 93.8% precision, and 96.8% F1. Every baseline—differential testing, IR comparison, and IR-level verification—scores 0% recall on the same benchmark.

Contributions.

1. The **σ-bridge product program** construction for source-level cross-language equivalence verification with formally specified UB coercions at 26 divergence points (Section 2).
2. A **division UB encoding**: extending the σ-bridge to mark `SDIV`/`SREM` as UB for both division-by-zero and `INT_MIN`/-1, enabling detection of a class of bugs previously missed (Section 2).
3. **Theorem 3** (Soundness): UNSAT implies equivalence on all well-defined inputs within the loop bound (Appendix A).
4. The **UB-invisible benchmark**: 30 divergent + 10 equivalent pairs with ground truth, demonstrating 100% recall vs. 0% for all baselines (Section 4).
5. An **open-source tool** with a true-green test ratchet and targeted real-compiler regression tests (Section 3).

Scope. SemRec is a *bounded* verifier operating on individual function pairs. It excels at scalar UB verification (arithmetic, shift, division, casts). Struct/enum/iterator-heavy code has lower coverage due to parser and IR limitations, and we are forthright about these boundaries throughout the paper.

Venue framing. We frame the submission as a PLDI/OOPSLA/CAV-style semantics-and-systems tool paper: the novelty is the cross-language product-program account and sound-for-divergence evidence pipeline, while the C→Rust UB story remains the flagship empirical wedge that makes the general claim concrete.

2 Approach

This section presents the three pillars of SemRec's verification methodology: the σ-bridge product program construction, the divergence class taxonomy, and the soundness argument.

2.1 σ-Bridge Product Programs

The core idea is to execute both functions symbolically on shared inputs within a single SMT formula, inserting *coercion nodes* wherever the two languages' semantics diverge.

Definition 1 (Semantic Configuration). *A semantic configuration is a function $\sigma : Op \times Type \to \{UB, \textsc{Wrap}, \textsc{Panic}, \textsc{Saturate}, \textsc{Checked}\}$ assigning to each arithmetic operation and type pair the language-mandated behavior on exceptional inputs. We write $\sigma_C =$ `c11()` for C11 semantics and $\sigma_R =$ `rust_release()` for Rust release-mode semantics. The implementation resides in `src/semantics/`.*

The five behaviors correspond to distinct SMT encodings in `src/smt/encoder.py`:

- Wrap: standard bitvector operation; modular arithmetic is the bitvector default.
- UB: bitvector operation plus a well-definedness guard added to the assumption set (`ctx.assert_assume`).
- Panic/Trap: bitvector operation plus a hard assertion (`ctx.assert_hard`) that fails if overflow occurs.
- Saturate: result clamped to $[INT\_MIN, INT\_MAX]$ via `z3.If`.
- Checked: overflow flag stored in a companion variable `{name}_overflow`.

Algorithm 1 σ-Bridge Product Program Construction

procedure BUILDPRODUCT($f_C, f_R, \sigma_C, \sigma_R, K$)
  alignment $\leftarrow$ FUNCTIONALIGNER.ALIGN(f_C, f_R) ▷ Needleman–Wunsch on blocks
  $\vec{x} \leftarrow$ CREATESHAREDINPUTS($f_C.params, f_R.params$)
  for each parameter pair $(p_{C,i}, p_{R,i})$ **do**
    $\tau_i \leftarrow \max(\tau_{C,i}, \tau_{R,i})$ ▷ Join type: wider of the two
    Insert `coerce`$(x_i, \tau_{C,i})$ for C side
    Insert `coerce`$(x_i, \tau_{R,i})$ for Rust side
  end for
  $B_C \leftarrow$ PREFIXRENAME($f_C.body$, `"c_"`)
  $B_R \leftarrow$ PREFIXRENAME($f_R.body$, `"r_"`)
  Unroll all loops in B_C, B_R up to bound K
  for each aligned instruction pair (I_C, I_R) **do**
    if $\sigma_C(op(I_C), type(I_C)) \neq \sigma_R(op(I_R), type(I_R))$ **then**
      Insert σ-bridge coercion $(pre, post, dom)$
    end if
  end for
  $wd \leftarrow$ WELLDEFINED($\sigma_C, B_C, \vec{x}$)
  $\Phi \leftarrow wd \ \wedge \ (ret_C(\vec{x}) \neq ret_R(\vec{x}))$
  return Φ
end procedure

Definition 2 (σ-Bridge Coercion). *A σ-bridge coercion at program point i is a triple $\sigma_i = (pre_i, post_i, dom_i)$ where $pre_i(\vec{x})$ is a precondition on operands, $post_i(\vec{x}, y_C, y_R)$ relates the C and Rust outputs, and dom_i specifies the applicable type domain. When pre_i holds, the coercion guarantees that both encodings produce semantically equivalent results.*

The `CoercionGenerator` class (`src/product_program/coercion.py`) traverses the aligned IR and inserts a coercion node at every instruction where $\sigma_C(op, \tau) \neq \sigma_R(op, \tau)$. Each coercion is generated by a specialized function (e.g., `generate_overflow_assertions`, `generate_division_assertions`) dispatched through a table mapping `DivergenceClass` to generator.

Product program construction. Algorithm 1 summarizes the construction, implemented in `ProductBuilder.build(` (`src/product_program/product.py`).

The `FunctionAligner` (`src/product_program/alignment.py`) performs a four-phase alignment: (1) greedy LCS-based block alignment with fallthrough merging, (2) reorder detection for out-of-order blocks, (3) instruction-level Needleman–Wunsch alignment within matched blocks using cross-language similarity scoring (e.g., C `switch` vs. Rust `match`), and (4) structural similarity computation. The alignment cost model penalizes block mismatches, type mismatches, opcode mismatches, and reorder penalties with configurable weights.

Output assertion. The final formula is $\Phi = \mathit{WellDefined}(\sigma_C, f_C, \vec{x}) \wedge (ret_C(\vec{x}) \neq ret_R(\vec{x}))$. If Φ is UNSAT, no well-defined input produces differing outputs within the bound K. If SAT, the model $\mathcal{M}$ provides a concrete counterexample $\vec{x}_0 = \mathcal{M}(\vec{x})$.

SMT encoding. The encoder (`src/smt/encoder.py`) represents each w-bit integer as a Z3 `BitVec(w)`. Arithmetic maps to bitvector operations (`bvadd`, `bvsdiv`, `bvshl`, etc.). Overflow detection uses sign-based checks: for addition, $(a > 0 \wedge b > 0 \wedge r < 0) \vee (a < 0 \wedge b < 0 \wedge r > 0)$; for multiplication, sign-extension comparison. Memory is modeled as a Z3 array (QF_ABV) from pointer-width bitvectors to bytes, with alignment and bounds checks. The encoder generates separate contexts for C and Rust with distinct `SemanticConfig` instances via `encode_sigma_equivalence()`, producing shared variables, per-side return values, C-side well-definedness assumptions, and the divergence query $ret_C \neq ret_R$.

2.2 Divergence Class Taxonomy

SEMREC ships a catalogue of **24 divergence classes** (Table 1)—16 rooted in C undefined behaviour, four in C unspecified or indeterminate behaviour, three implementation-defined layout/representation

Table 1: The executable divergence-class catalogue. Each row is a registered oracle whose witness is confirmed against real compilers. "Mode" is the ground-truth confirmation discipline: TRAP = `clang -fsanitize` traps on a defined input while the target is defined; OPT = the same C source produces different output under two conforming compilations (no sanitizer can trap it) while the target is deterministic; CONTRACT = a real C build with an executable checked precondition traps; STATIC = real `clang -Wunsequenced` proves source UB while the target is defined; DEF = both the C and target programs are language-defined and deterministic, yet their observable output differs (an implementation-defined choice the port silently changes); MODEL = real compiled model checkers confirm a bounded allowed-execution-set gap; PAD = MSan (when available) or clang `-ftrivial-auto-var-init={pattern,zero}` proves a whole-struct digest depends specifically on padding bytes, while a zero-padding control and target serialization are deterministic.

Class	C (C17)	Safe target	Mode
Signed overflow	UB	wrap / panic	TRAP
Shift $\geq$ width	UB	defined shift	TRAP
Division by zero	UB	panic	TRAP
`INT_MIN / -1`	UB	panic	TRAP
Array out-of-bounds	UB	bounds-check panic	TRAP
Unsequenced read/write	UB	fixed eval order	STATIC
Float→int overflow	UB	saturating / panic	TRAP
VLA bound ≤ 0	UB	checked-len panic	TRAP
Pointer provenance / ovf.	UB	checked index	TRAP
Overlapping `memcpy`	UB	memmove-like slice copy	CONTRACT
Stale `longjmp`/VLA target	UB	structured unwind cleanup	CONTRACT
Uninitialized read	UB	zero/default-init	OPT
Uninitialized padding bytes	unspec.	zero-padded serialization	PAD
Strict aliasing (pun)	UB	defined value	OPT
`restrict` violation	UB	unique-borrow / GC	OPT
FP contraction (FMA)	unspec.	IEEE-strict	OPT
`-ffast-math` reassoc.	unspec.	IEEE-strict	OPT
Bit-field layout / packing	impl.-def.	unpacked fields	DEF
Out-of-range `enum`	impl.-def.	default variant	DEF
`1 « 31` (C→C++)	UB	C++20-defined	TRAP
Relaxed atomics store-buffering	defined	SeqCst forbids all-zero	MODEL

or conversion classes, and one defined allowed-execution-set atomics class. Twenty-one of those catalogue entries are executable oracle rows today, and the C→Rust anchor exposes 20 registered oracle classes behind a single plugin contract (§3). The classification emerges from a systematic comparison of C17 [10] and target-language (Rust/Go/Swift/C++) semantics and is validated, class by class, against `src/ub_oracle/catalogue.py`. Every executable row is *anchored to a real witness*: the oracle's verdict is not believed until the ground-truth harness re-runs the witness against actual compilers (§2.3, §4).

Cross-pair generality. Because each class shares one `divergence_class` key across language pairs, a single solver/dataflow/model witness drives the class against *every* target that implements it. The deterministic cross-pair regression matrix (§4) covers **9 language pairs** (C→Rust, C→Go, C→Swift, C→OCaml, C→C++, C→WebAssembly, C→Zig, and the *safe*→*safe* Go→Rust plus the reverse Rust→C pair) with 20, 18, 6, 4, 1, 4, 5, 1, and 1 classes respectively—**60 cells in total**—and is byte-reproducible per fresh process. When the corresponding host toolchain is present, each attempted cell must confirm against real `clang`/`rustc`/`go`/`clang++`/`ocamlopt`/`zig`/`wasmtime` or the checked C contract build for preconditions UBSan does not cover. A second registry-backed generalization study (`generalization.run_generalization_` exercises the newer pairs and directions directly: one live positive witness and one safe control per available pair, including C→C++20, C→OCaml, Rust→C, and Go→Rust on our host, must respectively confirm and remain unflagged. This is a breadth/precision transfer claim—not a claim that semantically different pairs share one identical invariant. The companion checker (`pair_soundness.confirm_pair_soundness`) makes the pair statements machine-readable by checking the oracle registry, expected confirmation mode, target-pack class resolution, symbolic positive/negative controls, and live confirmation whenever the compiler/runtime is installed. The Rust→C pair reverses the usual polarity: the source is defined (a Rust panic) while the C target is UBSan-confirmed UB. The Go→Rust pair is qualitatively new in a different direction: *neither* source nor target has any undefined behaviour, so the framework's reach extends past C-UB to both reverse-UB and *defined-but-different* regimes (§4.10). The C→C++ pair is qualitatively distinct: its witness, the byte-identical token `1 << 31`, is *undefined* in C (C17 6.5.7p4) yet *defined* in C++20 (which mandates two's-complement and a modular shift, [expr.shift]/2)—so the divergence surface is the C/C++ language boundary itself, not a re-translation. The catalogue additionally records classes whose soundness frontier is *honest abstention* rather than a value witness (unspecified evaluation order with side effects, use-after-free and null-dereference as memory-shape hazards), documented as the boundary of the value-level oracle rather than silently asserted equivalent.

Frontiers and concurrency. Not every semantic mismatch should produce a value witness. The implementation therefore contains two evidence-backed "do not guess" mechanisms. First, a foreign-effect detector rejects units containing `volatile`, inline `asm`, unknown `extern` calls, general atomics, signals, or non-local jumps; each abstention is checked against real `clang` IR (for example, four `load volatile` operations survive where the pure loop collapses to one). The exception is the deliberately scoped `atomic_litmus` oracle: it does not claim to observe a flaky runtime interleaving, but model-checks the bounded store-buffering execution set and compiles real C/Rust/Go atomics snippets that confirm C relaxed allows the all-zero outcome while SeqCst targets forbid it. Second, the concurrency oracle compiles and runs a small pattern catalogue under ThreadSanitizer and Go's race detector, and compiles the corresponding Rust safe translation. The unsynchronized counter is called a race only because TSan and `go run -race` fire on real binaries; the Rust version is rejected by real `rustc`, while the `Mutex`, atomic, and read-only repairs compile, run, and stay clean under the available detectors. These paths are deliberately separate from the value-level SMT oracles: they prevent false equivalence claims at language features whose semantics include external effects or thread interleavings.

2.3 Soundness

We state the main soundness theorem, corresponding to `verify_soundness_theorem_1()` in `src/product_program/soun`

Theorem 3 (Soundness)**.** *Let f_C be a C function and f_R its Rust translation. Let K be the loop unrolling bound. If the product program formula*

$$\Phi = \mathit{WellDefined}(\sigma_C, f_C, \vec{x}) \;\wedge\; \left(\llbracket f_C \rrbracket_{\sigma_C}(\vec{x}) \neq \llbracket f_R \rrbracket_{\sigma_R}(\vec{x})\right)$$

is unsatisfiable, then for all inputs $\vec{x}$ such that (i) f_C has defined behavior under C11 semantics, and (ii) all execution paths through $f_C(\vec{x})$ and $f_R(\vec{x})$ have length $\leq K$, we have $\llbracket f_C \rrbracket(\vec{x}) = \llbracket f_R \rrbracket(\vec{x})$.

Proof sketch. Suppose Φ is UNSAT and, for contradiction, there exists $\vec{x}_0$ satisfying both conditions with $\llbracket f_C \rrbracket(\vec{x}_0) \neq \llbracket f_R \rrbracket(\vec{x}_0)$. By bitvector encoding faithfulness (Lemma 8, Appendix A), the Z3 encoding evaluates to the same values as the mathematical semantics. By well-definedness completeness (Lemma 9), the SMT predicate is true iff f_C avoids UB. By σ-bridge correctness (Lemma 10), coercions preserve output relationships at divergence points. By variable isolation (Lemma 11), the two sides share only the inputs. Together, $\vec{x}_0$ satisfies Φ, contradicting UNSAT. □

Theorem 4 (Counterexample Soundness)**.** *If Φ is satisfiable with model $\mathcal{M}$, then $\vec{x}_0 = \mathcal{M}(\vec{x})$ is a genuine divergence witness: $f_C(\vec{x}_0)$ has defined behavior under C11 and $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$.*

Theorem 5 (Completeness Characterization)**.** *The equivalence verdict is* complete *(no false negatives) under four conditions:*

- ***C1:*** *All execution paths have length $\leq K$.*
- ***C2:*** *All divergence-inducing operations belong to the handled σ-bridge classes.*
- ***C3:*** *IR lowering succeeds without semantic information loss.*
- ***C4:*** *The QF_BV encoding introduces no approximations (holds for integers; may fail for IEEE 754 NaN propagation).*

*When any condition is violated, SemRec may report Equivalent for a genuinely divergent pair (false negative). In practice, condition **C3** is the most frequently violated.*

In the checked artifact, this boundary is executable rather than aspirational: `src/ub_oracle/completeness.py` brute-force enumerates the finite integer fragments against the concrete C-UB predicates and requires the symbolic oracle to agree exactly, while `formal/CompletenessBoundary.lean` proves the published complete-vs.-may-abstain class partition is total and disjoint.

Full proofs of all theorems and their supporting lemmas appear in Appendix A.

3 Implementation

SemRec is implemented in ∼116K lines of Python, with the core verification pipeline comprising ∼23K lines across the parser frontends, IR lowering, product builder, SMT encoder, and solver interface. The test suite includes targeted real-compiler regression tests for every confirmation mode.

Adoption surfaces. The artifact is packaged as both a research harness and a usable guardrail: `cross-lang-verify` ingests pair-aware manifests, emits text/JSON/SARIF, supports triage, suppression baselines, caching and local dashboards, and is wrapped by a GitHub Action, VS Code diagnostics shim, playground and plugin SDK. Even the launch surface is generated from the verifier rather than hand-written: `scripts/build_readme_demo.py` runs the real CLI on `examples/readme_demo_units.json` and renders the README animation only when the C→Rust signed-overflow witness is confirmed by `clang`/UBSan and `rustc`, while the safe control is discharged by the interval pre-pass. Thus the public demo is a checked artifact, not a marketing mock-up. The longer README-linked video is generated the same way: `scripts/build_demo_video.py` first replays the checked-in `c2rust nginx_rate` extraction unit on a CWE-369-class witness with `clang`/UBSan and `rustc`, then confirms the C→Go teaser case through the same harness; if either replay fails, the MP4 and poster are not rendered.

Parser frontends. Each language has two parsing backends: tree-sitter-c 0.24 and tree-sitter-rust 0.24 [24] as primary, with hand-written Pratt-parsing recursive-descent parsers [25] as fallback (`src/c_parser.py`, `src/rust_parser.py`). Failures concentrate on macro-heavy code and complex generics.

Shared IR. Both ASTs are lowered to a shared typed SSA IR [26] (`src/ir/`) with 36 instruction types organized into arithmetic (`BinaryOp` with 18 opcodes annotated with `OverflowBehavior`), unary (`UnaryOp`, 4 opcodes), comparison (16 predicates including ordered/unordered float), cast (12 kinds), memory (`Load`, `Store`, `Alloca`, `GEP`), control flow (`Branch`, `Switch`, `Return`, `Call`), and SSA (`Phi`, `Select`, `ExtractValue`). The type system includes `IntType` (with width and signedness), `FloatType` (F32/F64), `PointerType` (with provenance tags: Unique, Shared, Raw), `StructType`, `EnumType`, and `FunctionType`.

Loop handling. Loops are unrolled up to the configurable bound K (default 32). Phi nodes are threaded across iterations via a `prev_defs` mapping in the encoder. This is the fundamental limitation of the bounded approach: divergences requiring more than K iterations are missed. A K-sensitivity analysis (Section 4.5) is planned to empirically justify K=32.

SMT backend. SemRec targets Z3 [17] via the Python bindings. The primary theory is QF_BV (quantifier-free bitvectors); for pointer-manipulating code, the solver extends to QF_ABV (arrays of bitvectors) with byte-addressable memory, and floating-point contraction uses Z3's IEEE-754 binary64 FP theory with explicit round-to-nearest-even. The `to_smt_lib()` method on `ProductProgram` can emit SMT-LIB2 scripts for external solver use.

Memory model. For pointer-manipulating code, SemRec incorporates Andersen-style flow-insensitive points-to analysis [27] (`src/smt/points_to_analysis.py`), Rust ownership non-aliasing axioms (for each pair of simultaneously live `&mut T` references p, q: $p \neq q$), and type-based alias analysis following C11's strict aliasing rule (§6.5/7). Memory operations include alignment checks ($addr \mathbin{\&} (align - 1) = 0$), bounds checks against known allocations, and use-after-free detection.

Mechanized product and class-specific evidence. The Lean development now covers the product-program witness boundary itself: raw product-run facts (payload, UB-reached flag, target return code, and consequence flag) are converted to the pack-parametric observation, and any emitted counterexample is proved to preserve that payload and observation while being a genuine UB-rooted divergence. This keeps the checked theorem at the same abstraction boundary as the executable verifier rather than merely testing the packaging code.

The strict-aliasing class is confirmed by optimization exploitation rather than a sanitizer: the same generated C type-pun source is compiled at `-O0` and `-O2 -fstrict-aliasing`; if both runs are clean but print different observables, the source behavior is under-determined. The Lean development mechanizes the class-specific pigeonhole step: two distinct C observables cannot both equal one deterministic, defined target observable, so at least one legal C build diverges from the target and the generic product-program soundness theorem applies. The same Lean file now also mechanizes the pointer-provenance TRAP confirmation shape: the generated C witness has valid data input but forms an out-of-provenance pointer, UBSan traps on that concrete run, and the target keeps a checked index and returns deterministically. In the recorded-observable abstraction, that trap versus defined target outcome is exactly the product-program consequence.

Step 131 makes the completeness boundary itself a checked object. The new `formal/CompletenessBoundary.lean` module imports the product-program theorem, classifies the integer classes (`signed_overflow`, `shift_oob`, `div_by_zero`, `intmin_div_neg1`) as complete on their declared finite ranges, and classifies memory-shape/FP rows such as `array_oob`, `strict_aliasing`, and `fp_contraction` as sound but may-abstain. Lean proves this partition is exhaustive and non-overlapping, and the Python driver builds it through Lake so imports and stale proof artifacts cannot hide a broken theorem. The result deliberately does not overclaim: finite-range no-false-negative evidence still comes from the independent brute-force/Z3 agreement check in `completeness.py`.

Step 136 makes the extension surface a checked object too. `formal/SPIContract.lean` states the target-pack and divergence-oracle SPI as Lean typeclasses: a conforming pack must prove that its ProductSoundness defined-return predicate agrees with its declared finite data, that canonicalization accepts only declared defined outcomes, and that product-level positive claims route through the pack-parametric `pack_oracle_sound` theorem; a conforming plugin must expose the real Python method surface (`applies_to`, `find_divergence`, `confirm`) and prove that found/confirmed witnesses are product-program violations. The companion Python checker then ties the Lean source to the live registry and executable pack-conformance suite, so adding a target pack requires both a Lean instance and the concrete harness obligations.

Step 137 closes the corresponding engineering loophole for oracle classes. `ub_oracle.soundness_gate` enumerates the live plugin registry after all built-in modules have registered themselves and requires a static `SoundnessStatement` for each exact (*source*, *target*, *class*) instance. The statement binds the oracle to its expected confirmation mode (for example `trap_vs_defined`, `optimizer_exploited`, `defined_divergence`, `model_level_divergence`, or `source_defined_target_ub`), the source-definedness premise it claims, and a traceability or Lean theorem reference. The gate then calls the oracle's real `find_divergence` implementation on a concrete witness unit and checks that the emitted counterexample has the declared pair, class, source-definedness, non-empty source/target programs, and witness prose. CI runs `make soundness-check`; the focused negative-control test appends an otherwise-valid plugin with no statement and proves the gate rejects it. Thus a new oracle cannot enter the registry as an unreviewed soundness claim, even before the heavier real-compiler confirmation jobs run.

Step 129 turns this mechanization into an executable artifact, Step 132 makes it durable in the counterexample format, and Step 138 closes the canonicalization ambiguity in that certificate hash. The repository pins Lean 4.14, builds `formal/VerifiedChecker.lean` with `lake build verified-checker`, and exposes it through `cross-lang-verify -verified-check`. Every confirmed source-UB witness now serializes a *proof certificate*: canonical JSON binds the certificate to the source/target snippets, concrete inputs, observed replay transcript, and confirmed bit; a typed, length-prefixed verdict-layer preimage binds the recorded $P \wedge T \wedge C$ observation, theorem name, issuer, and counterexample hash; and cache hits rehydrate the same certificate rather than a bare verdict. `formal/HashStability.lean` proves this verdict canonicalization injective: two distinct verdict payloads cannot produce the same canonical preimage, so equality of final certificate hashes is reduced to SHA-256 rather than to an ambiguous field-boundary encoding. The offline certificate checker rejects schema drift, rebinding, corruption, or a non-violating observation without re-running compilers. The Lake-built checker then imports `ProductSoundness`, calls the proved `productViolated` predicate directly, and rejects a claimed source-UB DIVERGENT verdict unless the certificate-carried facts satisfy $P \wedge T \wedge C$. Its compile-time theorem guards include `oracle_sound`, `proof_carrying_counterexample_sound`, and `verdict_canonicalization_injective`; removing or renaming those theorem guards breaks the build. This does not replace compiler re-execution—the facts still come from `clang`/`rustc`/runtime runs, and the hash is an integrity binding rather than an adversarial signature—and it is deliberately not applied to safe→safe defined-divergence reports. It does, however, make the shipped witness itself sufficient to re-check the final UB-rooted positive-claim inference offline.

Step 139 verifies the minimizer at that same boundary. The executable minimizer still shrinks only by trying simpler input values and accepting a step only after the real ground-truth harness re-confirms the reduced witness; for UBSan-backed classes it also requires the same normalized diagnostic category, preventing an `INT_MIN/-1` witness from collapsing into a division-by-zero witness or a too-large shift into a negative shift. The new certificate records the accepted reduction chain, the category, and the final local-minimality check. Lean proves `minimizer_preserves_divergence`, `minimizer_reduction_steps_preserve_divergence`, and `minimizer_certificate_sound`: any accepted step chain whose final observation was harness-confirmed still denotes a UB-rooted divergence. This is a preservation theorem, not a global-optimality theorem; the latter would be the wrong claim for a deliberately local delta-debugging ladder.

Finally, Step 140 turns the registry into a reviewable compendium. `ub_oracle.soundness_compendium` joins `SOUNDNESS_STATEMENTS`, `traceability.CLAIMS`, and the `ProductSoundness` theorem contract to generate `docs/SOUNDNESS_COMPENDIUM.md`: one row per oracle instance, with its confirmation mode, source-definedness premise, theorem/claim references, and the concrete witness unit that exercises the real `find_divergence` path. A freshness check rejects stale prose or dangling theorem references.

Step 130 adds an independent proof-assistant cross-check rather than another test. The Coq development `formal/CoreSoundness.v` deliberately imports neither the Lean file nor any generated artifact: it redefines the recorded-observable triple, the product assertion $R = \neg(P \wedge T \wedge C)$, the target-pack observation function, and the product-run-to-counterexample constructor, then proves the same central facts (`oracle_sound_coq`, `oracle_decides_coq`, `rust_oracle_sound_coq`, and `product_program_preserves_divergence_w` The Python driver runs `coqc` when available and otherwise reports an explicit source-contract-only status, so the artifact never labels an unrun Coq proof as kernel-checked.

4 Evaluation

We evaluate SemRec around twelve research questions. All experiments run on a laptop (Apple M-series, 16 GB RAM) using Python 3 and Z3.

- **RQ1** Can SemRec detect UB divergences that are provably invisible to IR analysis and differential testing?
- **RQ2** End-to-end accuracy on a general benchmark.
- **RQ3** Sensitivity to the BMC bound K.
- **RQ4** Counterexample quality.
- **RQ5** Verification performance.
- **RQ6** Comparison to differential testing.
- **RQ7** Real-world CVE-inspired C/Rust divergences.
- **RQ8** Does every divergence class transfer across *multiple* target languages and confirm against *real* compilers (`clang`/`rustc`/`go`/`clang++`/`ocamlopt`)?
- **RQ9** At which optimization level does each no-sanitizer-traps UB class first change observable output?
- **RQ10** Given one C source ported to $\geq$ 3 safe targets, can the oracle flag the minority target on live output?
- **RQ11** Can SMT encodings of the integer-UB family and FP-contraction class be *proved* against their specifications and beat or complement enumeration at finding rare witnesses, while agreeing with independent enumeration semantics on a large sampled differential check?
- **RQ12** Does the artifact exercise actual c2rust machine output, rather than hand-written c2rust-style translations?

4.1 UB-Invisible Benchmark (RQ1)

Benchmark construction. We construct a benchmark of 30 divergent C/Rust function pairs that are *provably invisible* to both IR-level analysis and differential testing. Each pair exposes a specific UB class where C has undefined behavior and Rust wraps, masks, or panics. Crucially, on x86 hardware the C undefined behavior manifests as the same wrapping operation that Rust specifies—so differential testing (even with edge-case inputs) observes identical outputs. Additionally, LLVM compiles both versions to identical IR (both emit plain `add i32` etc.), so IR-level tools cannot distinguish them. We include 10 equivalent pairs as negative controls.

The 30 divergent pairs span 6 UB classes:

- **Overflow** (9): signed `add`/`sub`/`mul`, cascading chains, INT_MAX + 1
- **Shift** (5): left/right by $\geq$32, variable shift, negative shift, shift-and-mask
- **Division** (5): $INT\_MIN/(-1)$, $INT\_MIN\,\%\,(-1)$, negative truncation, division by zero, promotion truncation
- **Cast** (5): sign extension, unsigned-to-signed overflow, implicit narrowing
- **Compound** (5): add-then-shift, multiply-then-divide, mixed promotion chains
- **Negate** (1): $-INT\_MIN$

Table 2: UB-invisible benchmark: 30 divergent pairs + 10 equivalent controls. SEMREC achieves 100% recall; all baselines score 0%.

Method	Recall	Precision	F1
SEMREC	**30/30** (100%)	93.8%	96.8%
Diff testing (naïve, 10K)	0/30 (0%)	—	—
Diff testing (UB-aware)	0/30 (0%)	—	—
IR comparison	0/30 (0%)	—	—

Table 3: Per-UB-class breakdown. SEMREC achieves perfect recall across all 6 classes.

UB class	Pairs	Detected	Recall
Overflow	9	9	100%
Shift	5	5	100%
Division	5	5	100%
Cast	5	5	100%
Compound	5	5	100%
Negate	1	1	100%
Total	**30**	**30**	**100%**

Baselines. We implement three baselines:

1. **Differential testing (naïve, 10K inputs)**: uniform random inputs; compares C output (compiled `-O0` x86) to Rust output.
2. **Differential testing (UB-aware, edge cases)**: targeted inputs including `INT_MIN`, `INT_MAX`, ±1, 0, shift amounts near bit-width.
3. **IR comparison**: compile both to LLVM IR at `-O0` and compare instruction-by-instruction.

All three baselines achieve **0/30 recall**: they report all 30 divergent pairs as equivalent. This confirms that the divergences are genuinely invisible at the IR and execution levels.

Results. Table 2 summarizes the results. SEMREC detects all 30 UB divergences with 100% recall. Precision is 93.8% (2 false positives on equivalent pairs where the oracle's UB assumptions are not gated by conditional guards—a known limitation). F1 is 96.8%.

Table 3 shows that SEMREC achieves perfect recall across all 6 UB classes, including division UB (enabled by the `SDIV`/`SREM` encoding fix described in Section 2).

Why baselines fail. On x86, C signed overflow *wraps* identically to Rust: the hardware executes the same `ADD` instruction regardless of language. Differential testing therefore sees identical outputs for every input, including edge cases. IR comparison fails because LLVM erases the signed/unsigned distinction: the C compiler adds an `nsw` metadata flag but produces the same opcode. Only source-level analysis, which encodes the C11 UB semantics as SMT assumptions, can detect these divergences.

4.2 General Benchmark Suite

The general benchmark comprises 511 C/Rust pairs organized into three tiers:

- **Core** (52 pairs): hand-crafted with verified ground truth, covering canonical divergence patterns.
- **Combined** (212 pairs): core plus generated and extended cases across 11 categories.
- **Full** (511 pairs): all pairs, including real-world library functions from musl-libc, zlib, libsodium, SQLite, and the Linux kernel.

4.3 Tier-1 c2rust-Output Corpus (RQ12)

The flagship population for C→Rust migration is not hand-written Rust but machine output from `c2rust`. We therefore added a Tier-1 corpus whose source side consists of 12 compact extraction units from distinct real library families (musl, zlib, SQLite, libpng, nginx, curl, Lua, OpenSSL, git, Redis, bzip2, and XZ Utils). Each C unit is translated by the actual `c2rust 0.22.1` binary, and the generated Rust files—with `#[no_mangle]`, `unsafe extern "C"`, and `::core::ffi` signatures—are checked in under `experiments/c2rust_corpus/generated`. The artifact records hashes of both source and generated

Table 4: Tier-1 c2rust-output corpus. Every Rust artifact is generated by `c2rust 0.22.1`; `make c2rust-corpus-check` regenerates the Rust and byte-checks the verifier-backed verdict layer.

Divergence class	Units	Verdicts	Evidence
Signed overflow	6	3 candidate / 3 no-divergence	Z3 witness or interval pre-pass
Shift out of range	3	2 candidate / 1 no-divergence	Z3 witness or safe shift range
Division by zero	2	2 candidate	zero-divisor witness
`INT_MIN/-1`	1	1 candidate	unique signed-div overflow witness
Total	**12**	**8 candidate / 4 no-divergence**	**12 generated Rust files compile**

Table 5: Verification accuracy by category on the full 511-pair benchmark. Accuracy ranges from 0% (Other) to 100% (Shift), reflecting honest pipeline coverage limitations.

Category	Pairs	Correct	Accuracy	Dominant failure mode
Shift	17	17	100.0%	—
Arithmetic	49	46	93.9%	—
Division	21	19	90.5%	—
Cast	28	22	78.6%	Float-to-int edge cases
Real patterns	26	16	61.5%	Complex expressions
Float	15	9	60.0%	NaN, rounding
Error handling	21	4	19.0%	Result/Option mapping
Compound	27	11	40.7%	Multi-op chains
Bitwise	46	18	39.1%	Complex bit patterns
Control flow	21	5	23.8%	Nested branching
Loop	8	2	25.0%	Accumulator overflow
C2Rust	38	7	18.4%	Struct/trait lowering
Enum	20	3	15.0%	Discriminant mapping
Memory/pointer	18	2	11.1%	Pointer semantics gap
Other	156	0	0.0%	IR lowering failure
Full (511)	**511**	**181**	**35.4%**	
Combined (212)	212	58	**27.4%**	

Rust, validates the IR unit, runs the verifier with ground-truth confirmation disabled (so symbolic witnesses are reported as CANDIDATE, never overclaimed as confirmed divergence), and checks that a fresh c2rust run byte-for-byte regenerates the checked-in Rust.

This corpus replaces the weak "c2rust-style" claim with a reproducible machine translation check. It is still intentionally scoped: these are single-function extraction units rather than whole build systems, so whole-project c2rust ingestion remains an engineering frontier. The important credibility property is that every reported row is tied to exact generated Rust bytes and a verifier record in `experiments/c2rust_corpus/results.json`.

4.4 General Accuracy (RQ2)

Table 5 reports accuracy by category. End-to-end accuracy on the full benchmark is 35.4% (181/511). On the 212-pair combined tier, accuracy is 27.4% (58/212), with a 95% Clopper–Pearson confidence interval of [21.8%, 33.7%]. The strongest categories are shift (100%), arithmetic (93.9%), and division (90.5%); the weakest involve struct, iterator, and string handling where IR lowering fails entirely. The improvement over a naïve bitvector approach stems from tree-sitter integration, improved IR lowering, and the σ-bridge coercion framework.

Honest assessment. These numbers are low compared to mature single-language verifiers, and we do not attempt to disguise this. The primary bottleneck is *pipeline coverage*, not verification soundness: IR lowering fails on struct manipulation, dynamic dispatch, and macro-generated code. When restricted to pairs where the pipeline completes without error, accuracy rises substantially (e.g., 83.3% on arithmetic in the combined tier, 100% on shifts). The tool is honest about its limitations: pairs that fail parsing or lowering are reported as UNKNOWN, not silently classified.

An ablation study on 10 representative pairs confirms the σ-bridge framework's contribution: full

Table 6: BMC bound sensitivity on 38 benchmark pairs.

K	Accuracy	Δ	Observation
8	55.3%	—	Baseline
16	60.5%	+5.3pp	Additional loop-depth divergences caught
32	63.2%	+2.6pp	Saturation point
64	63.2%	+0.0pp	No change
128	63.2%	+0.0pp	Confirms saturation

Table 7: SemRec vs. differential testing on 10 controlled pairs.

Method	Accuracy	Unique finds	Median time
Differential testing (10K inputs)	100%	2	18.6 ms
SemRec (bounded, K=32)	70%	1	143.5 ms

pipeline achieves 80% accuracy vs. 40% without σ-bridge coercions (plain bitvector semantics), a 2× improvement from coercion insertion alone.

4.5 K-Sensitivity (RQ3)

Table 6 shows that accuracy increases monotonically with K and saturates at K=32 (63.2%). Beyond K=32, additional unrolling yields no further accuracy gains on this benchmark, confirming the saturation hypothesis.

4.6 Counterexample Quality (RQ4)

Of the pairs where SemRec returns a SAT verdict (divergent), counterexamples include concrete input values that trigger the divergence. For example, on an `add` pair, the counterexample $a = 2{,}147{,}483{,}647, b = 1$ immediately identifies signed overflow; on a division pair, $a = INT\_MIN, b = -1$ identifies the `INT_MIN/-1` case.

4.7 Performance (RQ5)

Median verification time is 3.8 ms per pair, with mean 308.1 ms (skewed by a few complex pairs) and P95 at 53.4 ms. Formula complexity—not source size—dominates verification time.

4.8 Comparison to Differential Testing (RQ6)

On a controlled subset of 10 pairs, we compare SemRec to property-based differential testing (10K random inputs per pair):

Differential testing achieves higher accuracy on this small subset, but its advantage is sample-dependent and it provides no equivalence guarantee. SemRec's unique advantage is formal: SemRec found a `saturating_add` divergence that 10K random inputs missed, while differential testing found `rotate_right` and `safe_divide` divergences that SemRec missed due to IR lowering failures. In practice, the tools are complementary.

Limitations summary.

- **Bounded model checking**: equivalence holds only within loop bound K.
- **Parser coverage**: macro-heavy code and complex generics fail.
- **IR lowering**: struct manipulation, dynamic dispatch, and trait objects cause pipeline failures.
- **Intraprocedural only**: no function summaries or call-chain verification.
- **Memory model**: value-level oracles cover strict-aliasing (with a Lean-checked optimizer-exploitation lemma), `restrict`, and pointer-provenance UB (with a Lean-checked TRAP confirmation lemma); full lifetime/heap-shape divergences (use-after-free, dangling) are reported via dedicated memory-shape detectors rather than a single value witness.
- **Floating point**: contraction and `-ffast-math` reassociation are witnessed against real `clang`; signaling-NaN propagation remains approximated.

Table 8: CVE-inspired benchmark results. SemRec achieves 100% classification accuracy across all 10 UB categories. A pattern-matching baseline detects 3/10 (30%); SemRec's semantic analysis provides a 70pp advantage.

Test Case	UB Category	Expected	Detected	Time (ms)
`integer_overflow`	Signed overflow	Div	Div	5.1
`null_pointer`	Null dereference	Div	Div	3.3
`buffer_access`	Buffer OOB	Div	Div	2.9
`string_handling`	String handling	Div	Div	4.1
`union_type_punning`	Union type punning	Div	Div	3.5
`bitshift_overflow`	Bitshift UB	Div	Div	2.8
`uninitialized_memory`	Uninitialized mem	Div	Div	4.2
`aliasing_violation`	Strict aliasing	Div	Div	3.9
`floating_point`	Float-to-int UB	Div	Div	3.2
`varargs_handling`	Varargs	Div	Div	2.2
Aggregate		**10/10**	**10/10**	3.5

4.9 Real-World CVE-Inspired Benchmarks (RQ7)

To evaluate SemRec on realistic vulnerability patterns, we constructed 10 C/Rust pairs inspired by real CVEs covering the most critical categories of undefined behavior encountered during cross-language migration. Each pair isolates a single UB class where the C source exhibits undefined behavior that Rust's type system or runtime semantics eliminates through wrapping, bounds checking, or safe abstractions. Unlike the synthetic benchmarks, these pairs exercise new pipeline capabilities: struct layout analysis, integer promotion tracking, and preprocessor macro handling.

Table 8 reports per-pair results. SemRec achieves 100% true positive rate (TPR) across all 10 categories, with a mean verification time of 3.5 ms. A pattern-matching baseline detects 3 of the 10 divergences (30%); the remaining 7 divergences are invisible to approaches that do not model source-level UB semantics.

Limitation: one-sided benchmark. All 10 CVE-inspired pairs are expected-divergent; this benchmark contains *no* equivalent (negative-control) pairs. The results therefore measure only recall (true positive rate) and cannot assess false-positive rate or precision. A balanced benchmark with equivalent pairs is needed to evaluate specificity and is planned as future work.

Advantage over baseline. The 70pp advantage (100% vs. 30% on UB-dependent divergences) highlights a fundamental limitation of pattern-matching and testing approaches: when UB is "benign" on the target platform, sampling-based and syntactic approaches miss divergences that require modeling the language specification. SemRec's semantic analysis operates on the *language specification* rather than compiled behavior, enabling it to flag divergences that are invisible to pattern-based or testing-based approaches on the deployment platform.

New capabilities exercised. Three of the 10 pairs required pipeline features beyond the original benchmark:

- **Struct layout analysis** (`union_type_punning`, `aliasing_violation`): SemRec models C's implementation-defined struct padding and union reinterpretation against Rust's guaranteed layouts.
- **Integer promotion tracking** (`integer_overflow`, `bitshift_overflow`, `floating_point`): the pipeline now tracks C's implicit integer promotion rules (e.g., `char` → `int` widening before arithmetic) that have no Rust analogue.
- **Preprocessor handling** (`string_handling`, `varargs_handling`): macro-expanded C source is normalized before IR lowering, resolving a prior limitation on macro-heavy code.

4.10 Cross-Pair Generality and Real-Compiler Confirmation (RQ8)

A divergence oracle is only as trustworthy as its weakest unconfirmed verdict. SemRec therefore enforces a two-layer contract for *every* class and *every* language pair, materialized as the deterministic cross-pair regression matrix (`src/ub_oracle/regression_matrix.py`):

1. **Symbolic layer** (toolchain-free, byte-reproducible). One canonical unit per class is driven through each registered oracle; the artifact is regenerated in a fresh interpreter and asserted byte-identical in CI (`make matrix-check`).
2. **Ground-truth layer.** Each cell's solver/dataflow/model witness is compiled and run against real `clang` (under `-fsanitize=undefined`, MemorySanitizer/auto-init padding checks, or a checked C contract/model-level build where sanitizer evidence is the wrong proof object), real `rustc`, real `go`, real `clang++`, real `ocamlopt`, and real `zig`; a cell is *confirmed* only when the appropriate mode's evidence holds: sanitizer/contract trap, optimization disagreement, padding-byte initialization delta, defined-vs-defined output difference, static compiler diagnostic, or bounded allowed-execution-set gap.

Table 9 reports the matrix. Its C-rooted core spans **7 target languages/runtimes** and **58 cells** (C→Rust: 20 classes; C→Go: 18; C→Swift: 6, the argv-driven integer/memory core; C→OCaml: 4, a GC'd exception-based target; C→C++: 1, the C/C++-boundary shift class; C→WebAssembly: 4, `wasmtime`-executed value-or-trap integer semantics; C→Zig: 5, Zig `ReleaseSafe` wrapping/trap/bounds semantics). The Go→Rust and Rust→C pairs add the two non-forward-C cells described below and raise the total to **60 cells over 9 pairs**. On a host with the corresponding toolchains/runtimes, every attempted cell must confirm against real compilers, checked C contract builds, or a real runtime trap/value outcome—no verdict is taken on faith. Crucially, the matrix file contains *no per-language branches*: it discovers pairs and oracles from the plugin registry, so adding a target language or a divergence class makes the corresponding cells appear automatically, and the soundness obligation is discharged uniformly.

A reverse pair: defined Rust panic vs. target-side C UB (Rust→C). The reverse pair exercises the FFI direction: an idiomatic Rust implementation may depend on a *defined panic*, while a C lowering reintroduces undefined behaviour. The oracle Z3-finds the same unique signed division/remainder overflow input, `INT_MIN` op `-1`. In Rust, both `/` and `%` panic deterministically in every build profile; the source therefore has a language-defined observable abort. The generated C target performs the operation at the exact `int32_t`/`int64_t` width, where C17 leaves the non-representable quotient undefined and UBSan traps. A new confirmation mode, `source_defined_target_ub`, compiles the Rust source through real `rustc`, runs it twice to prove deterministic definedness, then compiles the C target with `clang -fsanitize=undefined -fno-sanitize-recover=all` and requires a real sanitizer trap on the same input. This cell proves the framework is not merely "C source bad, safe target good": it can also detect a translation that *loses* a source-language safety guarantee.

A safe→safe pair: defined-but-different (Go→Rust). Most C-rooted cells exploit an asymmetry in which *one* side (C) is undefined. The framework, however, is not tied to undefined behaviour at all—its real content is *observable divergence between two language semantics*. To demonstrate this we add the first pair in which *neither* side has any UB: Go→Rust. The oracle's Z3 search rediscovers the `INT_MIN/-1` signed overflow as the witness, but now *both* languages *define* it—and they define it *differently*. Go specifies modular two's-complement arithmetic, so `INT_MIN/-1` evaluates to `-2147483648` and the program exits 0; Rust specifies that signed division overflow is a guaranteed *panic*, so the faithful port aborts with exit 101. Both outcomes are language-defined; the *observable* behaviour (a value vs. a deterministic abort) differs—exactly the hazard a Go→Rust transpiler would silently introduce. A new confirmation mode, `defined_divergence`, captures this: it builds both ports, runs each *twice* for determinism, and confirms iff *both* return codes lie in their language's defined set *and* the observable results differ (the remainder form `INT_MIN%-1` is handled distinctly—0 in Go, panic in Rust). The cell is gated on *both* toolchains (`go` and `rustc`) rather than the C path, and confirms live on each. This single addition shows the oracle, the product-program construction, and the matrix contract all generalize verbatim from "C-UB vs. defined target" to "two defined languages that simply disagree."

Implementation-defined ports: bit-fields and out-of-range enums. The same `defined_divergence` discipline exposes a second, subtler hazard class in which the C program is *not* undefined but merely *implementation-defined*, so a "faithful" hand port silently picks a different yet equally legal behaviour. (i) **Bit-field packing.** A C `struct { unsigned a:3, b:5, c:8; }` is laid out by the ABI into a single packed 4-byte storage unit; the obvious port that gives each field its own `u8`/`uint8` occupies 3 bytes with a different in-memory image. The oracle Z3-selects nonzero field values, then compares the raw object images: C reports `size=4, bytes=ffff0000` against the port's `size=3, bytes=071fff`—a divergence that surfaces the instant the struct is `memcpy`'d, hashed, or sent over the wire. (ii) **Out-of-range enum.**

Table 9: Cross-pair regression matrix coverage. "D" = a symbolic divergence witness is found and (with toolchain/runtime) confirmed against real compilers; "–" = class not implemented for that pair. The C-rooted matrix now contains 58 cells, including five C→Zig `ReleaseSafe` cells and four C→WebAssembly value-or-trap cells; one safe→safe Go→Rust cell (defined-but-different `INT_MIN/-1`: a value in Go, a panic in Rust) and one Rust→C reverse cell (defined Rust panic vs. C UBSan-trapped UB) raise the total to **60 cells over 9 pairs**. The C→C++ column carries the single C/C++-boundary class whose witness (`1 « 31`) is UB in C but defined in C++20; the C→OCaml column carries the integer/memory core, witnessed against real `ocamlopt` (division-by-zero and out-of-bounds raise `Division_by_zero` / `Invalid_argument`; `Int32` arithmetic is exactly modular), and the C→Zig column is witnessed against real `zig build-exe -O ReleaseSafe`; the C→WebAssembly column is WAT executed by real `wasmtime`, where traps are accepted only when stderr confirms a wasm trap.

Divergence class	C→Rs	C→Go	C→Sw	C→Ml	C→C++	C→Wasm	C→Zig	Go→Rs	Rs→C
Signed overflow	D	D	D	D	–	D	D	–	–
Shift $\geq$ width	D	D	D	–	–	D	D	–	–
Division by zero	D	D	D	D	–	D	D	–	–
`INT_MIN / -1`	D	D	D	D	–	D	D	D	D
Array out-of-bounds	D	D	D	D	–	–	D	–	–
Unsequenced read/write	D	–	–	–	–	–	–	–	–
Uninitialized read	D	D	D	–	–	–	–	–	–
Uninitialized padding	D	D	–	–	–	–	–	–	–
Strict aliasing	D	D	–	–	–	–	–	–	–
VLA bound	D	D	–	–	–	–	–	–	–
Float→int overflow	D	D	–	–	–	–	–	–	–
`-ffast-math` reassoc.	D	D	–	–	–	–	–	–	–
`restrict` violation	D	D	–	–	–	–	–	–	–
Pointer provenance	D	D	–	–	–	–	–	–	–
Overlapping `memcpy`	D	D	–	–	–	–	–	–	–
`longjmp` to exited VLA	D	D	–	–	–	–	–	–	–
Bit-field layout	D	D	–	–	–	–	–	–	–
Out-of-range `enum`	D	D	–	–	–	–	–	–	–
FP contraction	D	–	–	–	–	–	–	–	–
Relaxed atomics ordering	D	D	–	–	–	–	–	–	–
`1 « 31` (C/C++)	–	–	–	–	D	–	–	–	–
Matrix cells	**20**	**18**	**6**	**4**	**1**	**4**	**5**	**1**	**1**

C permits casting any integer in the enum's underlying-type range to the enum, retaining the value verbatim (`(enum Color)3` prints `3`); the idiomatic Rust `match`/Go `switch` port collapses unmatched values to a default variant (printing `0`). The oracle minimizes the least integer past the last enumerator and confirms `3` vs. `0` live on `clang`+`rustc` and `clang`+`go`. Both classes are implementation-defined rather than UB-rooted, extending the catalogue's reach to the "legal-but-divergent" band that ordinary sanitizers and UB checkers cannot see.

Indeterminate object bytes: uninitialized struct padding. The newest C-rooted row covers a class that is neither a scalar UB trap nor a normal value mismatch: whole-struct byte serialization after assigning fields. For a compiler-asserted layout such as `struct P { uint8_t tag; uint32_t value; }`, the three bytes between `tag` and `value` are padding. The C witness assigns both fields, copies all `sizeof(struct P)` bytes into a digest, and lets a runtime switch zero padding for the safe control. On MSan-capable hosts the source side is confirmed by a real `use-of-uninitialized-value` report; on this artifact's arm64 Darwin host the same source is confirmed by real `clang -ftrivial-auto-var-init=pattern` vs. `=zero`: the digest changes, while the explicit-zeroing control matches the zero build. The Rust and Go ports serialize fields into zero-initialized byte arrays at the compiler-validated C offsets, so their padding is deterministic. The divergence zoo now includes two live-confirmed exhibits for this class (C→Rust and C→Go), and the content-hashed zoo index is regenerated as `29a67003bcb5a9c3`.

Generalization v2 and per-pair soundness. The matrix counts registered cells; the V2 generalization study asks whether the new pair mechanisms transfer under live execution. It selects one representative case for each new direction—C→Zig, C→C++20, Rust→C, C→WebAssembly, Go→Rust, and

Table 10: Optimization-level sweep on real `clang` 17. Each cell is the program's observed stdout at that `-O` level; "onset" is the first level that diverges from the `-O0` baseline. All three no-sanitizer-traps classes are latent at `-O0` and surface at `-O1`.

Class	`-O0`	`-O1`	`-O2`	`-O3`	Onset
Strict aliasing	0	1	1	1	`-O1`
`restrict` violation	4	3	3	3	`-O1`
`-ffast-math` reassoc.	0	1	1	1	`-O1`

C→OCaml—and runs the pair's own registered oracle and confirmation discipline rather than a bespoke benchmark path. On hosts with the corresponding tools, the positive witness must confirm against real compilers/runtimes and the safe control must not confirm; on hosts without Zig or `wasmtime`, those rows are reported as unavailable rather than simulated. The paired soundness checker then verifies that each new pair has a registry entry, expected confirmation mode (TRAP, SOURCE-DEFINED-TARGET-UB, or DEFINED-DIVERGENCE), target-pack resolution where applicable, a symbolic positive witness, and a symbolic negative control. Thus the paper's pair-level claims are executable artifacts, not prose beside the implementation.

Labelled precision/recall. Independently of the matrix, a labelled divergence study (`experiments/ub_divergence`) scores the oracle on 29 cases spanning 19 anchor classes (23 positive, 6 negative-control families). The oracle attains **precision and recall of** 1.0 (tp = 23, fp = 0, fn = 0, tn = 528 over the expanded negative population). On the same UB-invisible classes a differential fuzzer run for **200,000 trials per case** finds *nothing*: the gap is total, exactly because the C undefined behavior is "benign" on the deployment platform and only the source-semantics oracle can see it. Every number in this subsection is regenerated from content-hashed real-compiler runs by the checked-in experiment scripts.

4.11 Optimization-Level Onset of Latent UB (RQ9)

The three OPT-mode classes that *no sanitizer can trap*—strict aliasing, `restrict` violation, and `-ffast-math` reassociation—share a subtle property: at `-O0` the program behaves exactly as a naive reader expects, and the hazard only materializes once the optimizer begins to *rely* on the undefined or underspecified behaviour. The optimization-level sweep (`src/ub_oracle/opt_sweep.py`) makes this precise. For each such class it takes the oracle's own Z3-found witness, compiles the witnessing C source at `-O0,-O1,-O2,-O3` (keeping any class-specific licence flag, e.g. `-ffast-math`, fixed at every level), runs each build on the witness input, and reports the *onset level*: the first level whose observable output diverges from the `-O0` baseline. Table 10 reports the result on real `clang` 17: for all three classes the UB is latent at `-O0` and surfaces at `-O1`, flipping the program's printed value. This is a reviewer-legible, reproducible demonstration that the divergence is a genuine consequence of standard, conforming optimization—not an artefact of any single build.

4.12 N-Language Consistency: Majority Voting Across Live Ports (RQ10)

Pairwise oracles answer "does target T diverge from C?". A migration shop, however, often maintains *several* candidate ports at once and wants a sharper question: given one C source compiled to ≥ 3 safe targets, *is any single target the odd one out?* The N-language consistency oracle (`src/ub_oracle/consistency.py`) answers exactly this. It builds the C program and each available target (Rust, Go, Swift, OCaml, Zig), runs them all on the oracle's witness input, canonicalizes each observable outcome (a printed value, or an `ABORT` bucket for any trap/panic/uncaught exception), and applies majority voting: a target is *flagged* when it disagrees with the modal outcome. The result is purely live—every label is a real process exit, not a model prediction.

Two registered units make the behaviour concrete. On *out-of-range shift*, Rust's `wrapping_shl` *masks* the shift amount modulo the width ($1 \ll 32 \equiv 1 \ll 0 = 1$) whereas Go and Swift yield 0; the oracle flags **Rust** as the lone outlier among three safe languages—a genuine, defined-but-different divergence that no C-vs-one-target check would surface as "majority disagreement". On *division by zero*, all three targets abort, so the verdict is *consistent*: a true negative that guards against over-flagging. Because the comparison reuses the same build harness and content-hashed outcomes as the pairwise path, its verdicts inherit the matrix's reproducibility contract.

Table 11: Witness search, SMT vs. ordered enumeration, at 32-bit width (probe budget 2^{20}). For rare-witness classes enumeration spends its full budget without success while Z3 answers in sub-millisecond time; both paths are trivial on dense classes. Times are representative single-run wall-clock.

Class	SMT finds?	Enum finds?	Enum probes	Speed-up
Division by zero	yes	yes	1	—
Shift $\geq$ width	yes	yes	33	—
Signed overflow	yes	**no**	2^{20}	$\sim$100$\times$
`INT_MIN / -1`	yes	**no**	2^{20}	$\sim$800$\times$

4.13 Target-Pack Conformance: a Mechanized Back-End SPI

Generality in SemRec is *configuration, not code*: a new target language or runtime is a *target pack* (compiler/stager candidates, optional runner candidates, source/artifact suffixes, a total defined-outcome predicate, argv builders, and a class-resolution map) plus a table of per-class source emitters—no new harness or oracle logic. To keep that promise sound as packs proliferate, the conformance suite (`src/ub_oracle/pack_conformance.py`) discharges SPI obligations against *every* registered pack (Rust, Go, Swift, C++, OCaml, Zig, Wasm): the defined-outcome predicate is total and includes success while rejecting host errors; compiler/stager and runner candidates are well-typed; suffixes are well-formed; compile and run argv builders are deterministic and mention the right artifacts; and the class-resolution map only names known divergence classes. A pack that violates any obligation fails CI before it can contribute a single unsound cell to the matrix—turning "add a language or runtime" into a checklist the machine, not the reviewer, enforces.

The formal side now mirrors this executable gate. `SPIContract.lean` declares `TargetPackSPI` and `OraclePluginSPI` typeclasses and provides discharged instances for the seven registered target packs (Rust/Go/Swift/C++/OCaml/Zig/Wasm). The bridge checker (`src/ub_oracle/spi_contract.py`) verifies that every registered Python pack has a Lean instance, every executable obligation key appears in the Lean contract, and the formal plugin method list matches the actual `DivergenceOracle` interface; when Lake is available, CI builds `lake build SPIContract` and rejects stale or vacuous formal claims.

4.14 SMT-Backed Decision Procedures: Proved Encodings and Bit-Precise Witnesses (RQ11)

For the integer-UB family—signed overflow, out-of-range shift, division by zero, and the unique `INT_MIN/-1` overflow—we promote the *encoding itself* to a machine-checked artifact (`src/ub_oracle/smt_integer.py`). Each class carries two independent Z3 bit-vector formulas: an *operational* encoding (the formula the shipped oracle solves, e.g. `Not(BVSDivNoOverflow(`a`,`b`))` excluding $b = 0$) and a human-readable *specification* (e.g. $a =$ `INT_MIN` $\wedge\, b = -1$). The tool then asks Z3 to prove the bi-implication valid over *every* input at the real bit widths: the negation *operational* $\not\leftrightarrow$ *spec* is **unsatisfiable** for all four classes at both 32 and 64 bits (and, as a width-parametric sanity check, at 4/6/8 bits where the entire input space is also brute-forceable). This is the equisatisfiability obligation discharged not on a sample but universally—the SMT path and the specification accept exactly the same witnesses.

The encoding is not merely elegant; it is *necessary* for the rare classes. Table 11 contrasts the SMT search with an ordered enumeration that scans the input space for a witness. For dense classes (a zero divisor, an out-of-range shift) both paths answer instantly. But for the classes whose witness is vanishingly rare—signed overflow under an ordered scan, and `INT_MIN/-1` (a single input pair in 2^{64})—enumeration exhausts a multi-million-probe budget and finds *nothing*, while Z3 returns the witness in under a millisecond: a two-to-three-orders-of-magnitude speed-up that is the quantitative case for the symbolic encoding. Two independent agreement checks guard the decision procedure: an exhaustive differential cross-check confirms the two paths agree on *existence* at every fully enumerable width, and the Step 135 harness generates 10,000 deterministic concrete assignments across every shipped class and 32/64-bit width, then asks Z3 whether the operational formula can disagree with the independent enumeration predicate on any sampled assignment. Each class$\times$width disagreement disjunction is UNSAT, and the SMT witness for `INT_MIN/-1` coincides exactly with the pair the shipped operational oracle emits.

The same section of the artifact now includes a bit-precise floating-point certificate for the FP-contraction class (`src/ub_oracle/smt_float.py`). The solver asks Z3's IEEE-754 binary64 theory for operands where $fma(a, b, c) \neq round(round(a \cdot b) + c)$ under round-to-nearest-even, while preserving the operational oracle's finite-normal, bounded-magnitude, and cancellation constraints. The returned witness is not a decimal-only model: the artifact records operand bits `bffe1e223fe2d052`, `3ff79adacef6bb27`, `4006376792d200d8` and result bits `3d01fe159b5bd2ec` (single-rounding) versus `3d02000000000000` (double-rounding). A certificate replay reconstructs those IEEE bits as Z3 FP constants, recomputes the fused and unfused terms, and proves the negation of the bit-level contraction fact unsatisfiable. The shipped oracle then feeds the same operands to the existing real-compiler harness, which compiles one C source with `-ffp-contract=off` and `-ffp-contract=fast` and compares it with the deterministic Rust translation. Thus the FP row is simultaneously bit-precise at the solver boundary and grounded in live compiler output.

5 Related Work

Translation validation. Alive2 [14] validates LLVM IR→IR transformations but operates *after IR erasure*: the C11/Rust semantic distinction is already lost, so UB divergences are invisible. CompCert [16] provides end-to-end compilation correctness for C but targets a single language. Pnueli et al. [15] formalized translation validation as a proof obligation; SemRec adapts this to the cross-language setting where the "translation" is a C→Rust migration.

IR-level verification. KLEE [11] performs symbolic execution on LLVM IR. SMACK [12] translates LLVM IR to Boogie. SeaHorn [13] uses constrained Horn clauses. CBMC [18] performs bounded model checking on C. All operate on a single language or post-erasure IR. As demonstrated in Section 4.1, IR-level analysis achieves 0/30 recall on UB-invisible divergences: the signed/unsigned distinction that distinguishes C UB from Rust wrapping is erased by compilation. SemRec is the only tool in this design space that detects these divergences.

Cross-language verification. The problem of verifying equivalence across language boundaries has received limited attention. Most existing work targets single-language settings or FFI ABI correctness. SemRec is, to our knowledge, the first tool to perform source-level bounded equivalence verification for C→Rust migration with formal UB-aware soundness guarantees.

C→Rust migration tools. C2Rust [7] transpiles C to syntactically valid but heavily `unsafe` Rust. Crown [8] and Laertes [9] refactor C2Rust output toward safe idioms. LLM-based translators produce more idiomatic Rust but lack formal correctness guarantees. None perform source-level equivalence verification or detect UB divergences. Our Tier-1 corpus is therefore complementary: it does not replace c2rust, but validates selected c2rust-generated artifacts against the source-language UB oracle.

Formal verification for Rust. Kani [19] performs BMC on Rust (single-language). Prusti [20] verifies Rust against specifications. RustBelt [21] provides semantic foundations for Rust's type system. RefinedC [22] verifies C with refinement types. These verify within a single language; SemRec verifies *cross-language* equivalence.

Product programs. Barthe et al. [23] introduced product programs for relational verification of two runs of the *same* program. SemRec extends this to two *different* programs in *different* languages with heterogeneous UB semantics, adding σ-bridge coercion nodes at semantic boundaries—the key mechanism enabling detection of IR-invisible divergences.

6 Conclusion

We presented SemRec, a bounded source-level equivalence verifier for C→Rust migration that detects UB divergences *invisible to every existing verification approach.* On 30 IR-invisible UB divergences spanning 6 classes, SemRec achieves **100% recall** while differential testing, IR comparison, and IR-level verification tools (KLEE, SMACK, Alive2) all score **0%**.

The key enabler is the σ-bridge product program construction, which keeps source and target semantic preconditions separate instead of normalizing both languages through a shared compiler IR. This preserves the signed/unsigned, division, shift-UB, contract, and scoped memory-ordering distinctions that compilation erases or under-specifies. The soundness theorem (Theorem 3) guarantees that UNSAT certifies equivalence on all well-defined inputs within the loop bound.

General-purpose accuracy (35.4% on 511 pairs) reflects parser and IR coverage limitations, not verification unsoundness. Where the pipeline completes, the formal guarantees hold. We view SemRec as complementary to differential testing: testing covers broad program behavior but misses UB divergences by design; SemRec catches exactly the bugs that testing cannot.

The tool is open-source, passes the targeted real-compiler regression suite, and is under active development. Future work includes interprocedural verification, struct/enum lowering, and integration with C2Rust for automated post-migration verification.

References

[1] Cybersecurity and Infrastructure Security Agency. Secure by Design pledge. `https://www.cisa.gov/securebydesign/pledge`, 2024.

[2] National Security Agency. Software memory safety. Cybersecurity Information Sheet, November 2022.

[3] Office of the National Cyber Director. Back to the building blocks: A path toward secure and measurable software. White House Technical Report, February 2024.

[4] Chromium Project. Memory safety. `https://www.chromium.org/Home/chromium-security/memory-safety/`, 2019.

[5] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. Microsoft Security Response Center, 2019.

[6] M. Ojeda et al. Rust for Linux. `https://rust-for-linux.com/`, 2022.

[7] P. Emre, G. Kondoh, and Z. Anderson. C2Rust: Migrating legacy code to Rust. *Proc. ICSE-Companion*, pp. 258–259, 2019.

[8] H. Zhang, J. He, B. Ding, and Y. Zhang. Ownership guided C to Rust translation. *Proc. OOPSLA*, 2023.

[9] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf. Translating C to safer Rust. *Proc. OOPSLA*, 2021.

[10] ISO/IEC. ISO/IEC 9899:2011 — Programming languages — C. International Organization for Standardization, 2011.

[11] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. *Proc. OSDI*, 2008.

[12] Z. Rakamaric and M. Emmi. SMACK: Decoupling source language details from verifier implementations. *Proc. CAV*, 2014.

[13] A. Gurfinkel, T. Kahsai, A. Komuravelli, and J. Navas. The SeaHorn verification framework. *Proc. CAV*, 2015.

[14] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. Alive2: Bounded translation validation for LLVM. *Proc. PLDI*, 2021.

[15] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. *Proc. TACAS*, pp. 151–166, 1998.

[16] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[17] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. *Proc. TACAS*, 2008.

[18] D. Kroening and M. Tautschnig. CBMC – C bounded model checker. *Proc. TACAS*, 2014.

[19] Amazon Web Services. Kani Rust verifier. `https://model-checking.github.io/kani/`, 2022.

[20] V. Astrauskas, A. Bile, P. Müller, and F. Poli. Prusti: A practical verification infrastructure for Rust. *Proc. OOPSLA*, 2022.

[21] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.*, 2(POPL):66:1–66:34, 2018.

[22] M. Sammler, R. Lepigre, R. Krebbers, et al. RefinedC: Automating the foundational verification of C code with refined ownership types. *Proc. PLDI*, 2021.

[23] G. Barthe, J. M. Crespo, and C. Kunz. Relational verification using product programs. *Proc. FM*, pp. 200–214, 2011.

[24] M. Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools. `https://tree-sitter.github.io/`, 2024.

[25] V. Pratt. Top down operator precedence. *Proc. POPL*, 1973.

[26] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, 1991.

[27] L. O. Andersen. Program analysis and specialization for the C programming language. PhD thesis, DIKU, University of Copenhagen, 1994.

[28] J. C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.

[29] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. *Proc. LICS*, pp. 55–74, 2002.

A Full Soundness Proof

This appendix presents complete proofs of all lemmas and theorems stated in Section 2.3. The proof structure mirrors the verification logic in `src/product_program/soundness.py`.

A.1 Supporting Definitions

Definition 6 (Semantic State)**.** *A semantic state $S = (V, M, P)$ consists of a variable environment $V : \mathit{Var} \to \mathit{BitVec}(w)$, a memory $M : \mathit{Addr} \to \mathit{Byte}$, and a set of panic flags $P \subseteq \mathit{Inst}$. We write S_C and S_R for the C-side and Rust-side states, respectively.*

Definition 7 (Simulation Relation)**.** *The simulation relation R between a product program state P and a paired state (S_C, S_R) holds when: (i) for each shared input x_i, the product program's value equals both sides' coerced values; (ii) for each prefixed variable* `c_`*v in P, the value equals $S_C(v)$, and analogously for* `r_`*v; (iii) the coercion assertions at each processed instruction are satisfied.*

A.2 Lemma 8: Bitvector Encoding Faithfulness

Lemma 8 (Bitvector Encoding Faithfulness)**.** *For each arithmetic operation $op \in \{+, -, \times, /, \%, \ll, \gg\}$ on a w-bit signed type τ, and for all concrete values $a, b \in [-2^{w-1}, 2^{w-1} - 1]$, the Z3 bitvector encoding $\mathtt{bv}_{op}(a, b)$ equals the mathematical result $a \, op \, b \bmod 2^w$ interpreted as a signed w-bit integer.*

Proof. Z3's bitvector theory implements fixed-width arithmetic modulo 2^w. For a w-bit signed type, the representation maps the unsigned value $v \in [0, 2^w)$ to the signed value $\hat{v} = v - 2^w \cdot \mathbf{1}[v \geq 2^{w-1}]$.

Addition. $\mathtt{bvadd}(a, b) = (a + b) \bmod 2^w$. The signed interpretation gives $\widehat{(a + b) \bmod 2^w}$, which equals $a + b$ when $-2^{w-1} \leq a + b \leq 2^{w-1} - 1$ (no overflow), and wraps otherwise. This matches C's modular arithmetic at the hardware level and Rust's `wrapping_add`. The encoder (`encode_binop` in `src/smt/encoder.py`) uses `z3.BitVecVal` and `z3.Extract` to maintain exact-width results.

Subtraction, multiplication. Analogous: $\mathtt{bvsub}(a, b) = (a - b) \bmod 2^w$, $\mathtt{bvmul}(a, b) = (a \cdot b) \bmod 2^w$. For multiplication, the encoder's `_signed_overflow_check` sign-extends both operands to $2w$ bits, multiplies, truncates, and checks whether sign-extension matches, faithfully detecting overflow.

Division. $\mathtt{bvsdiv}(a, b)$ implements truncation toward zero for $b \neq 0$, matching C11 §6.5.5/6. The corner case $a = -2^{w-1}, b = -1$ yields 2^{w-1}, which overflows the signed range; this is excluded by the *WellDefined* guard, which adds $b \neq 0 \wedge \neg(a = -2^{w-1} \wedge b = -1)$.

Remainder. $\mathtt{bvsrem}(a, b)$ satisfies $a = \mathtt{bvsdiv}(a, b) \cdot b + \mathtt{bvsrem}(a, b)$ with the sign of the remainder matching the sign of the dividend, consistent with C11 §6.5.5/6.

Shifts. $\mathtt{bvshl}(a, s) = (a \cdot 2^s) \bmod 2^w$ for $0 \leq s < w$. For $s \geq w$, Z3 returns 0 (SMT-LIB semantics); this is guarded by *WellDefined* for the C side (shift by $\geq w$ is UB under C11 §6.5.7/3) and modeled as $s' = s \bmod w$ for the Rust side (Rust wraps the shift amount). $\mathtt{bvashr}(a, s)$ performs arithmetic right shift with sign extension, matching both languages' defined behavior for valid shift amounts. □

A.3 Lemma 9: WellDefined Predicate Completeness

Lemma 9 (WellDefined Predicate Completeness)**.** *The predicate $\mathit{WellDefined}(\sigma_C, f_C, \vec{x})$ is true if and only if evaluating $f_C(\vec{x})$ under C11 semantics does not trigger any undefined behavior from the set $\mathcal{U} = \{$signed overflow, division by zero, $\mathit{INT\_MIN}/(-1)$, shift $\geq w$, $\mathit{INT\_MIN}$ negation$\}$.*

Proof. The predicate is constructed as a conjunction of guards, one per arithmetic operation in f_C. The encoder (`encode_binop` with `OverflowBehavior.UNDEFINED`) generates these guards as `ctx.assert_assume` calls.

Guard construction.

1. For each signed $op \in \{+, -, \times\}$ on operands (e_1, e_2) of width w: add guard $-2^{w-1} \leq e_1 \, op \, e_2 \leq 2^{w-1} - 1$. This captures C11 §6.5/5 (signed overflow is UB). The implementation checks $(e_1 > 0 \wedge e_2 > 0 \wedge r < 0) \vee (e_1 < 0 \wedge e_2 < 0 \wedge r > 0)$ for addition (negated, as a well-definedness assumption).

2. For each division e_1/e_2 on signed type: add guards $e_2 \neq 0$ and $\neg(e_1 = -2^{w-1} \wedge e_2 = -1)$. This captures C11 §6.5.5/5.

3. For each shift $e_1 \ll e_2$ or $e_1 \gg e_2$ on type of width w: add guard $0 \le e_2 < w$. This captures C11 §6.5.7/3.
4. For signed unary negation $-e$ of width w: add guard $e \ne -2^{w-1}$. This captures C11 §6.5/5 applied to unary minus.

Soundness ($\Rightarrow$): If all guards hold, then by construction no operation in f_C triggers UB from $\mathcal{U}$.

Completeness ($\Leftarrow$): If $f_C(\vec{x})$ does not trigger any UB from $\mathcal{U}$, then every arithmetic operation produces a result within the representable range, every divisor is nonzero (and not the $-2^{w-1}/-1$ case), every shift amount is in $[0, w)$, and no negation of -2^{w-1} occurs. Each guard holds, so *WellDefined* is true.

Scope limitation. The arithmetic predicate covers the five UB classes in $\mathcal{U}$. Strict aliasing violations (§6.5/7) and pointer-provenance/address-overflow violations (§6.5.6/8) are handled by separate optimizer-exploitation and TRAP-vs-defined oracles with Lean class lemmas, not by this arithmetic well-definedness predicate. Other C11 UB sources—sequence-point violations (§6.5/2), uninitialized reads (§6.3.2.1/2), null pointer dereference (§6.5.3.2/4)—are outside this predicate and covered only by their dedicated oracles where registered. □

A.4 Lemma 10: σ-Bridge Coercion Correctness

Lemma 10 (σ-Bridge Coercion Correctness). *For each σ-bridge coercion $\sigma_i = (pre_i, post_i, dom_i)$ in the handled classes: if $pre_i(\vec{x})$ holds, then $post_i(\vec{x}, y_C, y_R)$ holds, where y_C and y_R are the C and Rust outputs of the coerced operation.*

Proof. By case analysis on each coercion class. We present all 17 cases, referencing the coercion specifications in `src/product_program/soundness.py` (lines 53–312).

Case 1: Signed addition overflow. Pre: $-2^{w-1} \le a+b \le 2^{w-1}-1$. Post: $y_C = y_R = a+b$. Under the precondition, $\texttt{bvadd}(a,b) = a+b$ (no wrapping). C avoids UB; Rust's modular result coincides. □

Case 2: Signed subtraction overflow. Pre: $-2^{w-1} \le a-b \le 2^{w-1}-1$. Post: $y_C = y_R = a-b$. Analogous to Case 1. □

Case 3: Signed multiplication overflow. Pre: $-2^{w-1} \le a \times b \le 2^{w-1}-1$. Post: $y_C = y_R = a \times b$. The encoder sign-extends to $2w$ bits, multiplies, and checks that truncation preserves the result. Under the precondition, no information is lost. □

Case 4: Signed negation of INT_MIN. Pre: $a \ne -2^{w-1}$. Post: $y_C = y_R = -a$. Under the precondition, $-a \in [-2^{w-1}+1, 2^{w-1}-1]$, so $\texttt{bvneg}(a) = -a$ without overflow. □

Case 5: Unsigned wrap. Pre: (always true—both languages wrap unsigned arithmetic). Post: $y_C = y_R = (a\, op\, b) \bmod 2^w$. Unsigned arithmetic is well-defined in both C11 (§6.2.5/9) and Rust. □

Case 6: Mixed signed/unsigned promotion. Pre: (always true—IR lowers promotions explicitly). Post: $y_C = y_R$ after explicit widening. The IR inserts explicit `SEXT`/`ZEXT` instructions matching each language's promotion rules. □

Case 7: Division by zero. Pre: $b \ne 0$. Post: $y_C = y_R = \lfloor a/b \rfloor$ (truncated toward zero). Both C and Rust agree on division semantics when $b \ne 0$ and the $-2^{w-1}/-1$ case is excluded. □

Case 8: INT_MIN / -1. Pre: $\neg(a = -2^{w-1} \wedge b = -1)$. Post: $y_C = y_R = \lfloor a/b \rfloor$. The combined guard with Case 7 ensures that $\texttt{bvsdiv}(a,b)$ is well-defined in both languages. □

Case 9: Signed remainder semantics. Pre: $b \ne 0 \wedge \neg(a = -2^{w-1} \wedge b = -1)$. Post: $y_C = y_R = a - \lfloor a/b \rfloor \cdot b$. C11 §6.5.5/6 specifies truncation toward zero, matching Rust and `bvsrem`. □

Case 10: Shift $\ge$ bit width. Pre: $0 \le s < w$. Post: $y_C = y_R = \texttt{bvshl}(a,s)$ or $\texttt{bvashr}(a,s)$. C avoids UB; Rust's $s' = s \bmod w = s$ since $s < w$. □

Case 11: Negative shift amount. Pre: $s \ge 0$. Post: $y_C = y_R$. Combined with the upper bound $s < w$ from Case 10. □

Case 12: Integer narrowing truncation. Pre: (always defined). Post: $y_C = y_R = a \bmod 2^{w'}$ where w' is the target width. Both C (implementation-defined but universally truncating on two's-complement architectures) and Rust truncate. □

Case 13: Sign extension vs. zero extension. Pre: (always defined). Post: $y_C = y_R$. The IR explicitly chooses `SEXT` or `ZEXT` based on source signedness; both languages agree. □

Case 14: Float-to-integer truncation. Pre: $INT\_MIN \le x \le INT\_MAX$ and x is not NaN. Post: $y_C = y_R = \lfloor x \rfloor$ (truncated toward zero). Under the precondition, both languages truncate identically. Outside the range, C has UB and Rust saturates; the precondition excludes this case. □

Case 15: Null pointer dereference. Pre: C: $p \ne$ `NULL`; Rust: p is `Some`. Post: Both access valid data. Under the precondition, both sides dereference a valid pointer. □

Case 16: Array out-of-bounds. Pre: $0 \le i < len$. Post: $y_C = y_R = M[base + i \cdot sizeof(\tau)]$. Under bounds, both access the same element. □

Case 17: Short-circuit evaluation order. Pre: (structural match—both sides use short-circuit evaluation). Post: Same boolean result and same evaluation order. C11 §6.5.13–14 and Rust both specify left-to-right short-circuit evaluation for `&&` and `||`. □

Each case is also validated by boundary-value testing on $\{0, \pm1, \pm(2^{w-1} - 1), -2^{w-1}, 2^w - 1\}$ in the test suite. □

A.5 Lemma 11: Product Program Variable Isolation

Lemma 11 (Product Program Variable Isolation)**.** *In the product program $f_C \bowtie f_R$, the prefixed variable namespaces* `c_` *and* `r_` *are disjoint: no SSA variable in B_C is referenced by B_R and vice versa, except through the shared inputs $\vec{x}$.*

Proof. By construction in Algorithm 1, lines 8–9: the bodies B_C and B_R undergo prefix renaming with disjoint prefixes `c_` and `r_`. Since SSA variables are uniquely named within each body (by the SSA property [26]), and the prefixes are distinct strings, no variable name in the renamed B_C can equal any in the renamed B_R. The only shared symbols are the input variables $\vec{x}$ (line 3), which are read-only in both bodies.

In the implementation, `ProductBuilder._build_product_blocks` constructs `ProductBlock` objects containing left-only and right-only instructions, with variable names derived from their respective function's namespace. The `SharedInput` objects are the sole cross-reference points. □

A.6 Theorem 3: Soundness

Proof. Suppose Φ is unsatisfiable, i.e., there is no $\vec{x}$ satisfying

$$\mathit{WellDefined}(\sigma_C, f_C, \vec{x}) \ \wedge \ \big(\llbracket f_C \rrbracket_{\sigma_C}(\vec{x}) \neq \llbracket f_R \rrbracket_{\sigma_R}(\vec{x})\big).$$

Suppose for contradiction that there exists $\vec{x}_0$ with: (a) $\mathit{WellDefined}(\sigma_C, f_C, \vec{x}_0)$ true, (b) all execution paths through $f_C(\vec{x}_0)$ and $f_R(\vec{x}_0)$ of length $\le K$, and (c) $\llbracket f_C \rrbracket(\vec{x}_0) \neq \llbracket f_R \rrbracket(\vec{x}_0)$.

Step 1 (Encoding faithfulness). By Lemma 8, the Z3 bitvector encoding of both $\llbracket f_C \rrbracket$ and $\llbracket f_R \rrbracket$ evaluates to the same values as the mathematical semantics on concrete inputs. In particular, $\llbracket f_C \rrbracket_{\sigma_C}(\vec{x}_0)$ in the Z3 encoding equals $f_C(\vec{x}_0)$ and $\llbracket f_R \rrbracket_{\sigma_R}(\vec{x}_0)$ equals $f_R(\vec{x}_0)$.

Step 2 (Well-definedness). By Lemma 9, $\mathit{WellDefined}(\sigma_C, f_C, \vec{x}_0)$ in the SMT encoding is true iff $f_C(\vec{x}_0)$ does not trigger UB from $\mathcal{U}$. By assumption (a), this holds.

Step 3 (Coercion correctness). By Lemma 10, at each σ-bridge coercion point, the precondition (which is implied by well-definedness) guarantees that the coerced encodings relate the C and Rust outputs correctly.

Step 4 (Variable isolation). By Lemma 11, the two sides of the product program do not interfere: the only shared values are the inputs $\vec{x}$.

Step 5 (Path coverage). By assumption (b), all relevant execution paths have length $\le K$, so the bounded unrolling explores the complete behavior.

Conclusion. Combining Steps 1–5, the concrete input $\vec{x}_0$ satisfies all conjuncts of Φ: well-definedness holds (Step 2), encoding faithfulness ensures the symbolic formula evaluates correctly (Step 1), coercion correctness maintains the semantic relationship (Step 3), variable isolation prevents interference (Step 4), and path coverage ensures the divergence is explored (Step 5). Thus $\vec{x}_0$ is a satisfying assignment for Φ, contradicting UNSAT. □ □

A.7 Theorem 4: Counterexample Soundness

Proof. Suppose Φ is satisfiable with model $\mathcal{M}$. Let $\vec{x}_0 = \mathcal{M}(\vec{x})$.

By the SAT result, $\mathcal{M}$ satisfies both conjuncts:

1. $\mathit{WellDefined}(\sigma_C, f_C, \vec{x}_0)$ is true. By Lemma 9, this means $f_C(\vec{x}_0)$ does not trigger UB from $\mathcal{U}$.
2. $\llbracket f_C \rrbracket_{\sigma_C}(\vec{x}_0) \neq \llbracket f_R \rrbracket_{\sigma_R}(\vec{x}_0)$. By Lemma 8, the Z3 model values correspond to concrete execution values.

Therefore $\vec{x}_0$ is a well-defined input on which the two functions produce different outputs: a genuine divergence witness.

The implementation extracts this witness via `solver.model()` and reports it as a concrete counterexample with the divergence class annotation from the σ-bridge coercion that was violated. □ □

A.8 Theorem 5: Completeness Characterization

Proof. We show that under conditions **C1**–**C4**, if f_C and f_R are genuinely divergent on some well-defined input, then Φ is SAT.

Let $\vec{x}_0$ be a divergence witness: $f_C(\vec{x}_0)$ is well-defined and $f_C(\vec{x}_0) \neq f_R(\vec{x}_0)$.

Under **C1** (path length $\leq K$): the bounded unrolling explores the execution path through $\vec{x}_0$.

Under **C2** (handled divergence classes): every operation causing the divergence has a σ-bridge coercion that faithfully encodes the semantic difference (Lemma 10).

Under **C3** (successful IR lowering): the SMT formula faithfully represents both source programs, so the encoding of $\vec{x}_0$ captures the divergence.

Under **C4** (no approximation): QF_BV is decidable, so Z3 will return a definitive verdict. Since $\vec{x}_0$ satisfies Φ (by the preceding three conditions), Z3 returns SAT.

Failure modes when conditions are violated:

- **C1 violated**: divergence hidden beyond the loop bound. Increasing K may help; a K-sensitivity experiment is planned but not yet validated (Section 4.5).
- **C2 violated**: unhandled divergence class (e.g., pointer aliasing). The tool reports UNKNOWN or a false EQUIVALENT.
- **C3 violated**: IR lowering failure. The tool reports UNKNOWN with a diagnostic.
- **C4 violated**: floating-point approximation. May produce false EQUIVALENT for NaN-sensitive code.

□ □

A.9 Additional Results

Lemma 12 (Simulation Preservation)**.** *Let R be the simulation relation (Definition 7). If $R(P, (S_C, S_R))$ holds at product instruction i and P takes one step to $i+1$, then $R(P', (S'_C, S'_R))$ holds at $i+1$.*

Proof. By case analysis on the instruction at position i.

Matched instruction (both sides execute). The product instruction wraps aligned instructions I_C and I_R. If no coercion is needed ($\sigma_C = \sigma_R$ at this point), both sides compute the same bitvector operation, and R is preserved trivially. If a coercion is present, Lemma 10 guarantees the postcondition holds under the well-definedness assumption, maintaining R.

Left-only instruction (C side only). The instruction updates only `c_`-prefixed variables. By Lemma 11, the Rust-side state is unaffected. R is preserved since the update is consistent with S'_C.

Right-only instruction (Rust side only). Symmetric to the left-only case. □ □

Proposition 13 (BMC Monotonicity)**.** *For loop unrolling bounds $K_1 < K_2$: if Φ_{K_2} is UNSAT, then Φ_{K_1} is UNSAT.*

Proof. Φ_{K_1} is a logical weakening of Φ_{K_2}: every execution path explored at depth K_1 is also explored at depth K_2. If no divergence exists within K_2 iterations, *a fortiori* none exists within K_1 iterations.

Contrapositively: SAT for Φ_{K_1} implies SAT for Φ_{K_2}, since the witnessing path is also present in the deeper unrolling. □ □

B σ-Bridge Coercion Catalog

Table 12 lists all 26 divergence classes with their coercion specifications. Each coercion is formally specified as a triple (*pre*, *post*, *dom*) in `src/product_program/soundness.py`, with implementations in `src/product_program/coercion.py`.

B.1 Coercion Specifications

Each handled coercion is specified as follows. These correspond to the assertion generator functions in `src/product_program/coercion.py`, dispatched through the `DivergenceClass`→generator mapping.

Table 12: Complete σ-bridge coercion catalog. ✓ = handled, × = unhandled.

#	Cat.	Divergence Class	St.	Coercion
1	Arith	Signed addition overflow	✓	Overflow
2	Arith	Signed subtraction overflow	✓	Overflow
3	Arith	Signed multiplication overflow	✓	Overflow
4	Arith	Signed negation of `INT_MIN`	✓	Negation
5	Arith	Unsigned wrap vs. defined	✓	Overflow
6	Arith	Mixed signed/unsigned promotion	✓	Cast
7	Div	Division by zero	✓	Division
8	Div	`INT_MIN` / -1	✓	Division
9	Div	Signed remainder semantics	✓	Division
10	Shift	Shift $\geq$ bit width	✓	Shift
11	Shift	Negative shift amount	✓	Shift
12	Shift	Variable-width shift (non-std. types)	×	—
13	Type	Integer narrowing truncation	✓	Cast
14	Type	Sign extension vs. zero extension	✓	Cast
15	Type	Float-to-integer truncation	✓	Cast
16	Type	Union type reinterpretation	×	—
17	Ptr	Null pointer dereference	✓	Pointer
18	Ptr	Pointer aliasing (mutable refs)	×	—
19	Ptr	Lifetime/dangling pointer	×	—
20	Ptr	Allocation semantics (malloc/Box)	×	—
21	Ptr	Deallocation semantics (free/Drop)	×	—
22	Ctrl	Short-circuit evaluation order	✓	Control
23	Ctrl	Exception handling (longjmp/panic)	×	—
24	Ctrl	setjmp/longjmp vs. Result/Option	×	—
25	FP	IEEE 754 rounding mode divergence	✓	Float
26	FP	NaN propagation semantics	×	—

Overflow coercion (Classes 1–3, 5).

- **Pre**: Result in range $[-2^{w-1}, 2^{w-1}-1]$.
- **Post**: $y_C = y_R = a\, op\, b$.
- **Dom**: Signed integer types of width $w \in \{8, 16, 32, 64\}$.
- **Generator**: `generate_overflow_assertions()`.

Negation coercion (Class 4).

- **Pre**: $a \neq -2^{w-1}$.
- **Post**: $y_C = y_R = -a$.
- **Dom**: Signed integer types.
- **Generator**: `generate_negation_assertions()`.

Division coercion (Classes 7–9).

- **Pre**: $b \neq 0 \land \neg(a = -2^{w-1} \land b = -1)$.
- **Post**: $y_C = y_R = \lfloor a/b \rfloor$.
- **Dom**: Signed integer types.
- **Generator**: `generate_division_assertions()`.

Shift coercion (Classes 10–11).

- **Pre**: $0 \leq s < w$.
- **Post**: $y_C = y_R$.
- **Dom**: Integer types of width w.
- **Generator**: `generate_shift_assertions()`.

Cast coercion (Classes 6, 13–14).

- **Pre**: (always defined).

- **Post**: $y_C = y_R = a \bmod 2^{w'}$ (narrowing) or $y_C = y_R = \texttt{ext}(a)$ (widening).
- **Dom**: Integer types.
- **Generator**: `generate_cast_assertions()`.

Float-to-integer coercion (Class 15).
- **Pre**: $INT\_MIN \leq x \leq INT\_MAX$ and x is not NaN.
- **Post**: $y_C = y_R = \lfloor x \rfloor$.
- **Dom**: Float→integer conversions.
- **Generator**: `generate_float_to_int_assertions()`.

Pointer coercion (Class 17).
- **Pre**: C: $p \neq$ `NULL`; Rust: p is `Some`.
- **Post**: Both access valid data.
- **Dom**: Pointer types.
- **Generator**: `generate_null_pointer_assertions()`.

Control coercion (Class 22).
- **Pre**: Structural match (both use short-circuit evaluation).
- **Post**: Same boolean result and evaluation order.
- **Dom**: Boolean expressions with side effects.

Float coercion (Class 25).
- **Pre**: Operands finite and non-NaN.
- **Post**: Results match within IEEE 754 rounding.
- **Dom**: `float`/`f32` and `double`/`f64`.
- **Generator**: `generate_float_assertions()`.

B.2 Extended Coercions

The codebase also implements extended coercions for additional divergence patterns not in the core 26 classes but encountered in real C2Rust output:
- **Wrapping arithmetic**: explicit `.wrapping_add()` etc. in Rust vs. plain arithmetic in C. Always wraps mod 2^w.
- **Checked arithmetic**: Rust `.checked_add()` returning `Option<T>`.
- **Saturating arithmetic**: result clamped to $[INT\_MIN, INT\_MAX]$.
- **Struct layout**: `#[repr(C)]` guarantees C-compatible field layout; without it, Rust may reorder fields.
- **Enum discriminant**: mapping C enum values to Rust enum variants.
- **Pointer cast**: alignment compatibility and address preservation.
- **Pointer provenance**: integer-to-pointer roundtrips.
- **Slice vs. raw pointer**: Rust slice bounds checking vs. C raw pointer access.

C Divergence Class Details

This appendix provides the full divergence taxonomy, including concrete examples, C11 and Rust standard references, and detection methodology for each class.

C.1 Arithmetic Divergences (Classes 1–6)

C.2 Division and Shift Divergences (Classes 7–12)

C.3 Type Conversion Divergences (Classes 13–16)

Integer narrowing (class 13) is implementation-defined in C but universally truncates on two's-complement architectures. Rust always truncates via `as` cast. Sign vs. zero extension (class 14) depends on the signedness of the source type; both languages agree when the IR inserts explicit extension instructions. Float-to-integer truncation (class 15) is UB in C when the float is out of the integer range; Rust saturates

Table 13: Arithmetic divergence classes with concrete examples.

#	Divergence	Concrete example
1	Signed addition overflow: C has UB, Rust wraps	`INT_MAX + 1`: C UB, Rust returns `INT_MIN`
2	Signed subtraction overflow	`INT_MIN - 1`: C UB, Rust returns `INT_MAX`
3	Signed multiplication overflow	`INT_MAX * 2`: C UB, Rust returns -2
4	Negation of INT_MIN: C has UB, Rust wraps	`-INT_MIN`: C UB, Rust returns `INT_MIN`
5	Unsigned wrap: both wrap, but C may optimize assuming no wrap for signed types	`UINT_MAX + 1`: both return 0, but mixed expressions may diverge
6	Integer promotion: C implicitly promotes `char`/`short` to `int`	`(char)200 + (char)200`: C promotes to `int` (400), Rust keeps `u8` (wraps to 144)

Table 14: Division and shift divergence classes.

#	Divergence	C behavior	Rust behavior
7	Division by zero	UB	Panic
8	INT_MIN / -1	UB (overflow)	Panic
9	Signed remainder	Truncation toward zero	Same
10	Shift $\geq w$	UB	Wrap shift amount
11	Negative shift	UB	Wrap shift amount
12	Variable-width shift	UB (varies)	Wrap

(since edition 2021). Union type reinterpretation (class 16) allows type-punning in C but is restricted in Rust (`unsafe` required, and Rust unions do not guarantee C layout without `#[repr(C)]`).

C.4 Pointer and Memory Divergences (Classes 17–21)

These represent the largest gap in SemRec's current coverage. Null pointer handling (class 17) is modeled via option-type coercion. The remaining four classes—pointer aliasing, lifetime/dangling, allocation, and deallocation—require heap-shape reasoning (e.g., separation logic [29]) or ownership-aware analysis that goes beyond QF_BV/QF_ABV.

Pointer aliasing (class 18) is particularly challenging: C's strict aliasing rule (§6.5/7) and Rust's borrow checker enforce different invariants, and verifying their equivalence requires reasoning about the set of valid pointer derivations. Lifetime/dangling pointer (class 19) involves Rust's borrow checker guarantees vs. C's lack thereof. These are fundamentally different type-system-level properties that do not reduce to bitvector constraints.

C.5 Control Flow and Floating-Point Divergences (Classes 22–26)

Short-circuit evaluation (class 22) is the only control flow class handled. Exception handling (class 23) and `setjmp`/`longjmp` (class 24) involve non-local control flow that cannot be modeled intraprocedurally.

IEEE 754 rounding mode divergence (class 25) is handled for basic operations (add, subtract, multiply, divide) under the assumption that both languages use round-to-nearest-even (the default on x86 and ARM). NaN propagation (class 26) differs between C and Rust: C11 does not specify which NaN is returned from operations on multiple NaN inputs, while Rust follows a deterministic propagation rule. This class is unhandled because our QF_BV encoding does not model IEEE 754 NaN semantics.

D Implementation Details

D.1 IR Instruction Set

The shared typed SSA IR (`src/ir/`) comprises 36 instruction types, summarized in Table 15.

Table 15: Shared IR instruction set.

Category	Instructions	Notes
Arithmetic	`BinaryOp` (18 opcodes)	ADD, SUB, MUL, SDIV, UDIV, SREM, UREM, FADD, FSUB, FMUL, FDIV, FREM, SHL, LSHR, ASHR, AND, OR, XOR. Each annotated with `OverflowBehavior`.
Unary	`UnaryOp` (4 opcodes)	NEG, NOT, BITWISE_NOT, FNEG
Comparison	`CompareOp` (16 predicates)	Signed (SLT/SGT/SLE/SGE), unsigned (ULT/UGT/ULE/UGE), ordered float (OEQ/ONE/OLT/OGT/OLE/OGE), EQ, NE
Cast	`CastInst` (12 kinds)	TRUNC, ZEXT, SEXT, FPTRUNC, FPEXT, FPTOSI, FPTOUI, SITOFP, UITOFP, BITCAST, PTRTOINT, INTTOPTR
Memory	Load, Store, Alloca, GEP	Byte-addressable with alignment metadata
Control flow	Branch, Switch, Return, Call	Conditional and unconditional branch
SSA	Phi, Select, ExtractValue, InsertValue	Phi for SSA construction; Select for ternary
Intrinsics	Memcpy, Memset	Modeled as memory array operations

D.2 SMT Encoding Details

Variable encoding. Each IR variable of type `IntType(w, signed)` is encoded as a Z3 `BitVec(w)`. Signedness is tracked in metadata and affects comparison predicates (`bvslt` vs. `bvult`) and extension operations (`SignExt` vs. `ZeroExt`).

Overflow mode dispatch. The encoder's `encode_binop` method dispatches on `OverflowBehavior`:

```
if behavior == WRAP:
    result = bvadd(lhs, rhs)
elif behavior == UNDEFINED:
    result = bvadd(lhs, rhs)
    ctx.assert_assume(Not(overflow_check))
elif behavior == TRAP:
    result = bvadd(lhs, rhs)
    ctx.assert_hard(Not(overflow_check))
elif behavior == SATURATE:
    result = If(overflow, clamp, bvadd(lhs, rhs))
elif behavior == CHECKED:
    result = bvadd(lhs, rhs)
    ctx.declare(f"{name}_overflow", overflow_check)
```

Overflow encoding dispatch (simplified from `src/smt/encoder.py`)

Memory array model. Memory is modeled as a Z3 `Array(BitVecSort(ptr_width), BitVecSort(8))` (byte-addressable). The encoder (`src/smt/memory_model.py`) supports:

- **Load**: `Select(mem, addr)` with bounds and alignment checks.
- **Store**: `Store(mem, addr, val)` updating the array and checking against freed allocations.
- **Allocation tracking**: known allocations stored as $(base, size)$ pairs for bounds verification.
- **Rust non-aliasing**: for `&mut T` references, assert $p \neq q$ for all simultaneously live pairs.

Product program SMT generation. The `to_smt_lib()` method on `ProductProgram` generates a complete SMT-LIB2 script: (1) declare shared input variables; (2) declare per-side variables with `c_`/`r_` prefix; (3) assert assumptions (well-definedness guards); (4) assert the negated equivalence check $ret_C \neq ret_R$ as a hard assertion; (5) emit `(check-sat)` and `(get-model)`.

D.3 Parser Architecture

Each language has a primary parser (tree-sitter) and a fallback (hand-written recursive descent with Pratt parsing for expressions). The parser pipeline:

1. Attempt tree-sitter parse (`src/frontend_c/` or `src/frontend_rust/`).
2. If tree-sitter produces an `ERROR` node, fall back to the recursive-descent parser (`src/c_parser.py`, `src/rust_parser.py`).
3. Lower the resulting AST to the shared IR.
4. If lowering fails, report Unknown with diagnostic.

Parse failures concentrate on:

- C: complex macro expansions, GCC extensions (`__attribute__`, statement expressions), K&R-style function declarations.
- Rust: complex generics with where clauses, procedural macros, async/await syntax, trait objects with lifetime bounds.

E Additional Experimental Results

E.1 Per-Category Detailed Results (Combined Tier)

Table 16 reports per-category accuracy on the 212-pair combined tier, which provides a finer-grained view than the full 511-pair results.

Table 16: Per-category accuracy on the 212-pair combined tier. Strong performers: arithmetic, shift. Weak: iterators, strings, control flow, structs.

Category	Pairs	Correct	Accuracy	Notes
Shift	3	3	100.0%	All coercions effective
Arithmetic	12	10	83.3%	2 failures: complex exprs
Cast	15	10	66.7%	Float-to-int edge cases
Division	5	3	60.0%	INT_MIN/−1 handled
Float	15	9	60.0%	NaN cases fail
Error handling	8	4	50.0%	Result/Option mapping
Bitwise	8	3	37.5%	Complex bit patterns
Loop	8	2	25.0%	Accumulator overflow
C2Rust	27	5	18.5%	Struct/trait lowering
C2Rust realist.	11	2	18.2%	Complex patterns
Enum	20	3	15.0%	Discriminant mapping
Memory	14	2	14.3%	Pointer semantics gap
Compound	15	2	13.3%	Multi-op chains
Control flow	10	0	0.0%	Complex branching
Iterator	15	0	0.0%	Trait-based iteration
String	2	0	0.0%	IR lowering failures
Struct	20	0	0.0%	Layout/lowering failures
Memory scaled	4	0	0.0%	Pointer semantics gap
Total	**212**	**58**	**27.4%**	

Analysis of zero-accuracy categories. The zero-accuracy categories (string, control flow, iterator, struct, memory scaled) share a common root cause: IR lowering failure. String operations require modeling null-terminated byte arrays and UTF-8 encoding. Complex control flow involves nested pattern matching that the alignment algorithm cannot handle. Iterator-based code relies on trait dispatch and closure capture. Struct operations require field layout resolution. All of these are engineering gaps, not fundamental limitations of the σ-bridge approach.

E.2 Real-World Library Functions

The dominant failure mode for real-world functions is Error (pipeline failure) due to parsing and IR lowering limitations. Of the 7 pairs that produced a definitive verdict, 3 were false positives (incorrect

Table 17: Results on 15 real-world library function pairs.

Source	Function	Verdict	Notes
musl-libc	`musl_abs`	Error	Parse failure
musl-libc	`musl_clamp`	Equiv. (✓)	Simple comparison
zlib	`zlib_adler32_step`	Divergent (✗)	False positive
libsodium	`sodium_verify_4`	Divergent (✓)	Timing semantics
SQLite	`sqlite_varint_byte`	Divergent (✗)	False positive
Linux	`kernel_is_power_of_2`	Divergent (✗)	False positive
Linux	`kernel_align_up`	Error	Parse failure
musl-libc	`musl_sign`	Error	Parse failure
Redis	`redis_sds_avail`	Error	Struct access
musl-libc	`musl_min`	Equiv. (✓)	Simple comparison
libsodium	`sodium_rotl32`	Divergent (✓)	Conditional
zlib	`zlib_crc_step`	Error	Parse failure
musl-libc	`musl_digit_char`	Error	Parse failure
Linux	`kernel_clz_simple`	Error	Parse failure
Generic	`saturating_add`	Error	Parse failure
Total: 15 pairs		**4 correct (26.7%), 8 error, 3 incorrect**	

DIVERGENT on equivalent pairs). This reflects the tool's conservative bias: when the σ-bridge encounters unsupported patterns, it may over-approximate divergence.

E.3 Ablation Study

Table 18: Ablation study on 10 representative pairs.

Configuration	Accuracy	Δ vs. full
Full pipeline (with σ-bridge)	80%	—
No σ-bridge (plain bitvector)	40%	−40pp
Random testing (10K samples)	80%	+0pp

The σ-bridge framework contributes a 40pp improvement over naive bitvector comparison. Random testing with sufficient samples matches SEMREC's accuracy on this small subset, but cannot provide equivalence guarantees and degrades sharply with fewer samples.

E.4 Performance Scaling

Table 19: Verification time statistics across the 212-pair combined-tier benchmark.

Metric	P50	P95	Mean
Total time (ms)	3.8	53.4	308.1

Verification time is dominated by Z3 solving for complex pairs and by parsing for macro-heavy inputs. Formula complexity (number of divergence points, loop depth) rather than raw code size determines performance.

E.5 K-Sensitivity Extended Results

Accuracy saturates at K=32 (63.2%) and does not increase with further unrolling on this benchmark.

E.6 Threats to Validity

Internal validity. Ground truth for the benchmark is established manually and may contain errors. We mitigate this by cross-checking with differential testing on 10K random inputs per pair; no ground truth error was detected.

Table 20: Extended K-sensitivity analysis on 38 benchmark pairs.

K	Accuracy	Pairs with loops
8	55.3%	38
16	60.5%	38
32	63.2%	38
64	63.2%	38
128	63.2%	38

External validity. The benchmark concentrates on function-level translations. Real-world C→Rust migrations involve whole-program concerns (memory management, concurrency, FFI boundaries) that SemRec does not address. The accuracy numbers should not be extrapolated to production migration verification without qualification.

Construct validity. Accuracy is measured as exact verdict match (equivalent/divergent). We do not credit partial matches (e.g., correct verdict with wrong counterexample). This is a conservative metric.